STOCKWHISK ENTERPRISE SUITE

MASTER SUITE

Complete System Documentation & QA Specification

Full 15-Volume Production Reference Handbook for StockWhisk SaaS Platform

DOCUMENT SUITE	TARGET ARCHITECTURE	RELEASE VERSION	STATUS
15 Modules & Specs	Django REST + Next.js 14	v1.4.0 Production	✅ Verified & Approved

📚 Master Suite Table of Contents

STOCKWHISK SAAS PLATFORM

DOC-01

Software Requirements Specification (SRS)

IEEE 830 Standard Functional & Non-Functional Requirements Specification

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Technical Architecture	Core Engineering, QA, System Architects	v1.4.0 Production	✅ Verified & Approved

Software Requirements Specification (SRS) for StockWhisk

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for StockWhisk, a multi-tenant SaaS Inventory, Point of Sale (POS), Warranty, and Financial Management system designed primarily for retail and service businesses in Bangladesh.

1.2 Scope

The StockWhisk platform encompasses the following primary features:
- **Inventory Tracking:** Real-time tracking of stock levels, including per-unit serial lifecycle tracking.
- **Point-of-Sale (POS):** Fast and efficient checkout with barcode scanning, multiple payment methods, and EMI scheduling.
- **Customer & Supplier Management (CRM):** Tracking customer profiles, due balances, supplier accounts, and purchase histories.
- **Warranty & Service Repairs:** Managing warranty claims, verifying warranty status, and tracking repair service tickets.
- **Financial Accounting:** Ledger-based accounting for cash flows, expenses, daily settlements, and investment tracking.
- **Multi-branch Operations:** Stock transfers and branch-level reporting.
- **Platform Administration:** Billing, subscription management, tenant onboarding, and platform configurations.

1.3 Tech Stack

- **Backend:** Django REST Framework (Python 3.x)
- **Frontend:** Next.js 14 (React, TypeScript, Tailwind CSS)
- **Database:** PostgreSQL (production), SQLite/MySQL support
- **Async Tasks:** Celery + Redis for background processing
- **Deployment:** Docker, Docker Compose (with Caddy reverse proxy)

2. Overall Description

2.1 Product Perspective

StockWhisk is a cloud-hosted Software as a Service (SaaS). It employs strict tenant isolation at the application level using a thread-local context and a `TenantScopedModel` base class. This ensures data security and privacy for each shop without requiring separate database schemas, simplifying maintenance and scaling.

2.2 User Classes

- **Platform Super Admin:** Platform owner/operator managing subscriptions, tenants, and system-wide settings.
- **Shop Owner:** Primary account holder for a tenant, with full access to their shop's data and billing.
- **Shop Manager:** An employee with broad permissions configured by the Shop Owner to oversee daily operations.
- **Cashier:** A frontline staff member primarily interacting with the POS, sales, and basic customer operations.
- **Reseller:** A third-party partner who refers new shops to StockWhisk and tracks commissions via a dedicated portal.

2.3 Operating Environment

- **Client Side:** Modern web browsers (Chrome, Firefox, Safari, Edge). Responsive design for desktop, tablet, and mobile viewing.
- **Server Side:** VPS deployment using Docker Compose. Components include a Caddy reverse proxy for SSL, the Django backend application, Celery workers/beat for scheduled tasks, and the PostgreSQL database.

2.4 Design Constraints

- **Multi-tenant Data Isolation:** Must strictly enforce tenant isolation within a single database using `tenant_id` foreign keys and thread-local context filtering.
- **Auditability:** Financial data (LedgerEntry) must be append-only to maintain strict audit trails and prevent historical data tampering.
- **Authentication:** Stateless JWT (JSON Web Tokens) for API authentication.

3. Functional Requirements

3.1 Authentication & Authorization (FR-AUTH)

- **Registration:** Phone number or email registration with OTP verification.
- **Login:** Secure JWT-based login.
- **Password Reset:** OTP or email link based password recovery.
- **Permissions:** Role-based access control (RBAC) governing access to specific modules and actions.

3.2 Catalog Management (FR-CATALOG)

- **Product CRUD:** Create, read, update, delete products.
- **Categorization:** Manage categories, brands, and units of measurement.
- **Serial Tracking:** Manage `ProductUnit` items for individual serial numbers.
- **Inventory Toggle:** `track_inventory` boolean to differentiate between physical goods and services.
- **Barcodes:** Automatic or manual barcode generation for products.

3.3 Point of Sale (FR-POS)

- **Interface:** Fast, keyboard-friendly interface for cart management.
- **Scanning:** Barcode scanner integration for rapid item addition.
- **Payments:** Multi-payment checkout support (Cash, bKash, Nagad, Card, Bank).
- **Quotation:** Ability to save carts as quotations without deducting stock.
- **EMI:** Direct integration for setting up EMI schedules from the checkout screen.

3.4 Sales Management (FR-SALES)

- **Invoices:** Listing, viewing, and generating PDF invoices for completed sales.
- **Sharing:** Direct WhatsApp sharing of invoices.
- **Payments:** Adding subsequent payments to open or partially paid invoices.
- **Corrections:** Mechanisms for authorized users to correct sales records (typically via returns/adjustments).

3.5 Returns (FR-RETURNS)

- **Processing:** Handling sale returns with options to restock items.
- **Refunds:** Processing cash or digital refunds.
- **Exchanges:** Managing product replacements within the return workflow.

3.6 Inventory Management (FR-INVENTORY)

- **Ledger:** Detailed `StockMovement` ledger tracking all inventory changes.
- **Adjustments:** Manual stock adjustments (loss, damage, audit corrections).
- **Alerts:** Low-stock notifications based on configurable thresholds.
- **Lifecycle:** Tracking the state of individual serialized units (IN_STOCK → SOLD → DEFECTIVE → RETURNED_SUPPLIER).

3.7 Purchasing (FR-PURCHASING)

- **Suppliers:** Managing supplier profiles and contact information.
- **Purchase Orders:** Creating POs for stock replenishment.
- **Receiving:** Processing received POs to increment stock levels.
- **Payments:** Tracking payments to suppliers, including credit purchases and outstanding balances.

3.8 Customer Relationship Management (FR-CRM)

- **Profiles:** Maintaining customer details and purchase history.
- **Due Balances:** Tracking outstanding due amounts for credit sales.
- **Collections:** Processing due collections seamlessly without artificially inflating total sales revenue.
- **History:** Comprehensive view of all customer interactions and payments.

3.9 EMI Management (FR-EMI)

- **Scheduling:** Creating EMI schedules based on down payment, interest rate, and duration.
- Formula: Principal = Total - DownPayment; Interest = Principal × Rate/100; Monthly = (Principal+Interest)/Months
- **Tracking:** Recording and tracking individual installment payments and statuses.

3.10 Accounting & Finance (FR-ACCOUNTING)

- **Expenses:** Tracking operational expenses categorized by custom `ExpenseCategory`.
- **Ledger:** Strict, append-only `LedgerEntry` for all cash flow tracking.
- **Settlements:** `DailySettlement` for shift closings, including denomination counting.
- **Investments:** Tracking capital additions and drawings.
- **Transfers:** Inter-account fund movements (e.g., Cash to Bank).
- **Reporting:** Profit & Loss (P&L) statements and Financial Position reporting.

3.11 Warranty & Service (FR-SERVICE)

- **Verification:** Checking warranty status via serial number or invoice (ACTIVE / EXPIRING_SOON / EXPIRED).
- **Ticketing:** Managing repair service tickets with states (RECEIVED → DIAGNOSING → AWAITING_PARTS → IN_REPAIR → READY → DELIVERED / CANCELLED).
- **Parts:** Tracking parts used in repairs, with optional inventory stock deduction.
- **Claims:** Managing claims made to suppliers or manufacturers.

3.12 Reports & Analytics (FR-REPORTS)

- **Dashboards:** HIGH-level sales and profit overviews.
- **Analytics:** Dead stock analysis, top-performing products.
- **Filtering:** Comprehensive date-range and criteria filtering.
- **Exports:** Exporting report data in CSV and PDF formats.

3.13 Settings & Configuration (FR-SETTINGS)

- **Profile:** Shop details, logo, address, and VAT/Tax configurations.
- **Toggles:** Feature toggles to enable/disable modules (e.g., hiding EMI if not used).
- **Staff Management:** Inviting users and assigning roles/permissions.

3.14 Billing & Subscriptions (FR-BILLING)

- **Plans:** Managing SaaS subscription tiers and features.
- **Payments:** Uploading manual payment proofs (e.g., bKash TRX ID, Bank deposit slips) for admin verification.
- **Trials:** Managing free trial periods and expirations.

3.15 Notifications (FR-NOTIFICATIONS)

- **In-App:** Real-time alerts for system events.
- **Digests:** Automated low-stock email/SMS digests.
- **Reminders:** Automated EMI and due payment reminders.
- **WhatsApp:** Configuration for automated WhatsApp messaging.

3.16 Multi-branch Operations (FR-BRANCHES)

- **Transfers:** Initiating and receiving stock transfers between different physical branch locations.

3.17 Reseller Portal (FR-RESELLERS)

- **Tracking:** Reseller portal for tracking referred shops, subscription payments, and calculating commissions.

4. Non-Functional Requirements

- **Performance:** API endpoints must respond within 200ms at the 95th percentile. POS operations must feel instantaneous to the cashier.
- **Security:** HTTPS everywhere. Strict tenant isolation. Protection against common web vulnerabilities (XSS, CSRF, SQLi via ORM).
- **Scalability:** Stateless backend design allows horizontal scaling of Django application servers.
- **Availability:** Target 99.9% uptime. Automated daily database backups.
- **Usability:** Intuitive UX tailored for high-speed retail environments.
- **Internationalization (i18n):** Full support for English and Bangla interfaces and messaging.

5. External Interface Requirements

- **User Interfaces:** Web-based, responsive UI built with Next.js and Tailwind CSS.
- **API Interfaces:** RESTful JSON APIs documented via Swagger/OpenAPI for potential third-party integrations.
- **Hardware Interfaces:** Support for standard USB/Bluetooth barcode scanners (acting as keyboard input) and thermal receipt printers (via browser print dialog or specialized print drivers).

STOCKWHISK SAAS PLATFORM

DOC-02

Complete Module & App Catalog

Comprehensive Inventory of 22 Domain Modules, Models, and APIs

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Architecture Catalog	Developers, Tech Leads, Integrators	v1.4.0 Production	✅ Verified & Approved

Complete Module List

This document provides a detailed inventory of all Django apps (modules) within the StockWhisk system.

1. tenants

- **Description:** Core multi-tenancy management. Handles the creation and configuration of isolated shop environments.
- **Key Models:** `Shop` (Tenant entity), `Branch` (Physical locations), `Subscription` (Tenant plan details).
- **Key API Endpoints:** `/api/tenants/shops/`, `/api/tenants/branches/`
- **Dependencies:** None (Core dependency for almost all other modules).
- **Status:** Active

2. accounts

- **Description:** User authentication, authorization, and profile management.
- **Key Models:** `User` (Custom auth model), `Role` (RBAC group), `Permission` (System capabilities), `Registration` (OTP/Sign-up flow).
- **Key API Endpoints:** `/api/accounts/login/`, `/api/accounts/register/`, `/api/accounts/roles/`
- **Dependencies:** `tenants`
- **Status:** Active

3. catalog

- **Description:** Product information management (PIM). Defines what is sold.
- **Key Models:** `Product`, `Category`, `Brand`, `Unit`, `ProductVariation` (Size/Color), `ProductUnit` (Unique serial instances).
- **Key API Endpoints:** `/api/catalog/products/`, `/api/catalog/categories/`
- **Dependencies:** `tenants`
- **Status:** Active

4. inventory

- **Description:** Tracks the movement and stock levels of products across branches.
- **Key Models:** `StockMovement` (Ledger of additions/deductions).
- **Key API Endpoints:** `/api/inventory/movements/`, `/api/inventory/adjust/`
- **Dependencies:** `tenants`, `catalog`
- **Status:** Active

5. crm

- **Description:** Customer Relationship Management.
- **Key Models:** `Customer` (Profile & balances), `CustomerPayment` (Due collections).
- **Key API Endpoints:** `/api/crm/customers/`, `/api/crm/payments/`
- **Dependencies:** `tenants`
- **Status:** Active

6. purchasing

- **Description:** Procurement and supplier management.
- **Key Models:** `Supplier`, `PurchaseOrder`, `PurchaseOrderItem`, `PurchasePayment`, `SupplierPayment`.
- **Key API Endpoints:** `/api/purchasing/suppliers/`, `/api/purchasing/orders/`
- **Dependencies:** `tenants`, `catalog`, `inventory`
- **Status:** Active

7. sales

- **Description:** Core sales processing and historical invoice tracking.
- **Key Models:** `Sale` (Invoice), `SaleItem`, `Payment` (Transaction), `SaleReturn`, `SaleReturnItem`, `EMISchedule`, `EMIInstallment`.
- **Key API Endpoints:** `/api/sales/invoices/`, `/api/sales/returns/`, `/api/sales/emi/`
- **Dependencies:** `tenants`, `catalog`, `crm`, `inventory`, `accounting`
- **Status:** Active

8. pos

- **Description:** Highly optimized views tailored specifically for the fast-checkout Point of Sale interface.
- **Key Models:** (Relies on `sales` models).
- **Key API Endpoints:** `/api/pos/checkout/`, `/api/pos/cart/`
- **Dependencies:** `sales`, `catalog`, `crm`
- **Status:** Active

9. accounting

- **Description:** General ledger and financial tracking.
- **Key Models:** `Expense`, `ExpenseCategory`, `RecurringExpense`, `LedgerEntry` (Append-only cash flow), `DailySettlement`, `Investment`, `AccountTransfer`.
- **Key API Endpoints:** `/api/accounting/expenses/`, `/api/accounting/ledger/`, `/api/accounting/settlements/`
- **Dependencies:** `tenants`
- **Status:** Active

10. analytics

- **Description:** Aggregation views for dashboards and data visualization.
- **Key Models:** None (Uses raw SQL/ORM aggregations on other models).
- **Key API Endpoints:** `/api/analytics/dashboard/`, `/api/analytics/sales-summary/`
- **Dependencies:** `sales`, `inventory`, `accounting`
- **Status:** Active

11. billing

- **Description:** SaaS platform subscription billing and invoice management.
- **Key Models:** `SubscriptionInvoice`, `ManualPayment` (Proof uploads).
- **Key API Endpoints:** `/api/billing/invoices/`, `/api/billing/upload-payment/`
- **Dependencies:** `tenants`
- **Status:** Active

12. branches

- **Description:** Logistics for moving physical goods between different shop branches.
- **Key Models:** `StockTransfer`, `StockTransferItem`.
- **Key API Endpoints:** `/api/branches/transfers/`
- **Dependencies:** `tenants`, `inventory`, `catalog`
- **Status:** Active

13. notifications

- **Description:** Alerting system for users (In-app, SMS, Email, WhatsApp).
- **Key Models:** `Notification`, `ShopAlertConfig`, `ShopWhatsAppConfig`.
- **Key API Endpoints:** `/api/notifications/list/`, `/api/notifications/config/`
- **Dependencies:** `tenants`, `accounts`
- **Status:** Active

14. service

- **Description:** Warranty claim processing and repair ticketing system.
- **Key Models:** `Warranty`, `WarrantyClaim`, `ServiceTicket`, `ServiceTicketPart`, `ServiceTicketStatusHistory`.
- **Key API Endpoints:** `/api/service/tickets/`, `/api/service/warranty/`
- **Dependencies:** `tenants`, `catalog`, `sales`, `inventory`
- **Status:** Active

15. resellers

- **Description:** Partner portal for tracking referrals and commissions.
- **Key Models:** `ResellerProfile`, `ResellerCommission`.
- **Key API Endpoints:** `/api/resellers/profile/`, `/api/resellers/commissions/`
- **Dependencies:** `accounts`, `tenants`, `billing`
- **Status:** Active

16. public_api

- **Description:** External developer API access layer.
- **Key Models:** `APIKey`.
- **Key API Endpoints:** `/api/public/v1/...`
- **Dependencies:** Core models across apps.
- **Status:** Planned/Active

17. platform_admin

- **Description:** Super admin tools for managing the entire SaaS platform.
- **Key Models:** `PlatformConfig`, `TutorialVideo`, `BlogPost`, `SoftwareRelease`, `ShopDataBackup`.
- **Key API Endpoints:** `/admin/...` (Django Admin or custom internal API)
- **Dependencies:** `tenants`, `billing`
- **Status:** Active

18. reports

- **Description:** Dedicated views for generating and exporting complex CSV/PDF reports.
- **Key Models:** (Relies on models across apps).
- **Key API Endpoints:** `/api/reports/sales/`, `/api/reports/inventory/`
- **Dependencies:** `sales`, `inventory`, `accounting`, `crm`
- **Status:** Active

19. scanner

- **Description:** Utilities supporting mobile barcode scanner integration.
- **Key Models:** N/A
- **Key API Endpoints:** `/api/scanner/process/`
- **Dependencies:** `catalog`, `sales`
- **Status:** Active

20. imports

- **Description:** Bulk data ingestion utilities (e.g., importing legacy catalogs).
- **Key Models:** N/A (Tracks import jobs via Celery).
- **Key API Endpoints:** `/api/imports/csv/`
- **Dependencies:** `catalog`, `crm`
- **Status:** Active

21. web

- **Description:** Handles server-rendered templates (e.g., public landing pages, invoice PDFs, password reset emails).
- **Key Models:** N/A
- **Dependencies:** Various
- **Status:** Active

22. core

- **Description:** Foundational utilities and middleware, particularly for multi-tenancy.
- **Key Components:** `TenantScopedModel` (Base model ensuring tenant isolation), `TenantManager`, `TenantMiddleware` (Extracts tenant from request), `tenant_context` (Thread-local storage).
- **Dependencies:** None
- **Status:** Active

STOCKWHISK SAAS PLATFORM

DOC-03

Role & Granular Permission Matrix

Enterprise Access Control, Role Hierarchies, and Security Scopes

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Security & Governance	Security Auditors, Admins, Developers	v1.4.0 Production	✅ Verified & Approved

Role and Permission Matrix

StockWhisk utilizes a Role-Based Access Control (RBAC) system to govern what users can see and do within the platform.

1. Roles Definition

1. **Platform Super Admin:** Platform owner/operator. (`is_staff=True` , `is_superuser=True`). Has unrestricted access to all platform configurations, billing, and all tenant data across the entire system.
2. **Shop Owner:** Primary account holder for a specific tenant. (`is_owner=True`). Has unrestricted access to their specific shop, including billing, subscription management, and user/role administration.
3. **Shop Manager:** A configurable role intended for senior staff. Can be customized by the Shop Owner, but typically has broad access to manage inventory, sales, reports, and basic settings.
4. **Cashier:** A frontline role. Typically restricted to creating sales via the POS, viewing basic product info, and perhaps processing returns. Restricted from sensitive financial reports or backend configuration.
5. **Reseller:** A platform partner. (`is_reseller=True`). Access is strictly limited to the Reseller portal to view their referrals and commissions; they do not have access to any specific shop's internal data.

2. Permission Matrix

The following table maps core permissions (from the `accounts.Permission` model) to standard roles.

(Note: Shop Owner has all permissions implicitly. Manager and Cashier permissions are highly configurable by the Shop Owner; the below represents a typical default configuration.)

PERMISSION KEY	DESCRIPTION	PLATFORM ADMIN	SHOP OWNER	SHOP MANAGER (TYPICAL)	CASHIER (TYPICAL)	RESELLER
`view_dashboard`	View main analytics dashboard	✅	✅	✅	❌	❌
`view_products`	View product catalog	✅	✅	✅	✅	❌
`manage_products`	Create/Edit/Delete products	✅	✅	✅	❌	❌
`view_inventory`	View stock levels	✅	✅	✅	✅	❌
`manage_inventory`	Adjust stock, transfers	✅	✅	✅	❌	❌
`create_sale`	Access POS, generate invoices	✅	✅	✅	✅	❌
`view_sales`	View past invoices	✅	✅	✅	✅	❌
`edit_sales`	Modify existing invoices	✅	✅	❌	❌	❌
`process_return`	Handle sales returns	✅	✅	✅	✅	❌
`view_customers`	View customer CRM profiles	✅	✅	✅	✅	❌
`manage_customers`	Edit/Delete customers, debts	✅	✅	✅	❌	❌
`manage_purchasing`	Handle POs and Suppliers	✅	✅	✅	❌	❌
`view_reports`	Access financial/sales reports	✅	✅	✅	❌	❌
`manage_expenses`	Record operational expenses	✅	✅	✅	❌	❌
`view_service`	View warranty/service tickets	✅	✅	✅	✅	❌
`manage_service`	Edit/Update service tickets	✅	✅	✅	❌	❌
`manage_users`	Add staff, change roles	✅	✅	❌	❌	❌
`manage_settings`	Edit shop profile, VAT, configs	✅	✅	❌	❌	❌

PERMISSION KEY	DESCRIPTION	PLATFORM ADMIN	SHOP OWNER	SHOP MANAGER (TYPICAL)	CASHIER (TYPICAL)	RESELLER
`view_accounting`	View ledgers, P&L	✅	✅	✅	❌	❌
`manage_accounting`	Alter ledgers, investments	✅	✅	❌	❌	❌

3. Implementation Details

3.1 Backend Enforcement (Django DRF)

Permissions are enforced at the API level using custom Django REST Framework permission classes (e.g., `HasPermission('create_sale')`).
- The middleware attaches the authenticated `User` to the request.
- The permission class checks if the user's assigned `Role` contains the required `Permission` for the current tenant context.
- Owners (`is_owner=True`) automatically bypass these checks for their own shop.

3.2 Frontend Enforcement (Next.js)

The frontend uses a `can(permission_key)` helper function provided by an authentication context or hook.
- UI elements (buttons, menu links) are conditionally rendered based on the result of `can()`.
- Route guards prevent unauthorized access to specific pages, redirecting users if they lack required permissions.

3.3 Role Management

- Roles are created and managed by the Shop Owner within the "Settings > Roles" area.
- The Owner defines a Role (e.g., "Senior Cashier") and toggles specific permissions for that role.
- Users (staff) are then invited to the shop and assigned one of these custom roles.

3.4 Demo Mode Restrictions

The system includes a `DemoReadOnlyMiddleware`. If a shop is flagged as a "Demo Shop", this middleware automatically intercepts and blocks unsafe HTTP methods (`POST`, `PUT`, `PATCH`, `DELETE`), returning a 403 Forbidden. This allows users to safely explore the platform without altering demo data, regardless of their assigned role permissions within that demo shop.

3.5 Impersonation

Platform Super Admins have the ability to "impersonate" a shop. This allows them to log in and view the application exactly as the Shop Owner would, aiding in customer support and troubleshooting. This action is logged securely in the system audit trails.

STOCKWHISK SAAS PLATFORM

DOC-04

Business Rules & Calculation Specification

Mathematical Formulations for Accounting, EMI, COGS, Stock & P&L

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Financial Specification	Accounting Specialists, Core Engineers	v1.4.0 Production	✅ Verified & Approved

Business Rules & Calculation Specifications

This document outlines all core business logic, calculation formulas, and state transition rules for StockWhisk.

1. Cash Flow & Accounting Rules

The financial engine operates on a strict double-entry ledger-like system tailored for retail operations.

- **Golden Rule:** `Expected Cash = Opening Balance + Σ Inflows - Σ Outflows`
- **Inflows:**
  - Sales payments (cash/digital received)
  - Due collections
  - Capital investments (Owner contributions)
- **Outflows:**
  - Purchase payments to suppliers
  - Operating expenses
  - Sale refunds
  - Owner drawings
- **COGS Rule:** Cost of Goods Sold (COGS) is snapshotted at the time of sale (`SaleItem.unit_cost`). It is NEVER deducted again from cash on sales. Profit is calculated as `Sale Price - COGS` .
- **Account Transfer Rule:** Moving money between internal accounts (e.g., Cash → bKash → Bank) does NOT affect P&L. It only rebalances liquid accounts via offsetting LedgerEntries.
- **Drawing Rule:** Owner withdrawals reduce equity/capital and liquid cash, NOT operating expenses. They do not affect the P&L statement.
- **Immutability:** Ledger entries are append-only and immutable. Corrections require a reversing entry to maintain a strict audit trail.

2. Sales & POS Rules

- **Visibility:** Zero-stock products are excluded from the POS grid (filter: `current_stock > 0`), unless `track_inventory=False`.
- **Cart Total Calculation:**
  `Cart total = Σ(item.price × qty) - discount + delivery_charge + VAT`
- **Sale Status:**
  - `PAID`: `paid ⩾ total`
  - `PARTIAL`: `0 < paid < total`
  - `DUE`: `paid == 0`
- **Payments:** A single sale can accept multiple payment methods (Cash, bKash, Nagad, Card, Bank) simultaneously.
- **Quotation Mode:** Saves an invoice document without deducting stock or affecting accounting ledgers.

3. EMI Calculations

Equated Monthly Installment (EMI) sales follow these standard formulas:

- **Principal:** `Total Invoice Amount - Down Payment`
- **Interest:** `Principal × (Interest Rate % / 100)`
- **Total EMI Amount:** `Principal + Interest`
- **Monthly Installment:** `Total EMI Amount / Number of Months`

4. Purchase Rules

- **Cash Purchase:** Immediately reduces liquid funds (via LedgerEntry) and increases stock.
- **Credit Purchase:** No immediate cash deduction. Increases stock and increases Supplier Payable (`due_balance`).
- **Receiving PO:** Increments `Product.current_stock` via a stock movement function `apply_movement(PURCHASE_IN)`.
- **Serialization:** If a product is serialized, receiving a PO creates individual `ProductUnit` records for each item quantity.

5. Returns & Exchanges Rules

- **Return with Restock:** Stock is added back via `apply_movement(SALE_RETURN_IN)`.
- **Serialized Return:** The specific `ProductUnit` status reverts to `IN_STOCK`, `DEFECTIVE`, or `RETURNED_SUPPLIER` based on condition.
- **Refund:** Creates a negative `LedgerEntry` to deduct from the respective cash/bank account and adjusts total sales.
- **Exchange:** Swaps a defective unit (moves to `TESTING_PENDING`) for a new `IN_STOCK` unit. Rolls over the existing warranty to the new unit.

6. Due Collection Rules

- **Nature:** Due collection is NOT a new sale; it is debt settlement.

- **Accounting:** Increases liquid funds (positive `LedgerEntry`).
- **Customer:** Decreases `Customer.due_balance`.
- **Sale Record:** Updates the specific `Sale` paid amount and recalculates its status (e.g., from `PARTIAL` to `PAID`).

7. Settlement/Shift Closing Rules

Designed for daily register reconciliation.

- **Expected Cash Calculation:** `Expected Cash = Opening Cash + Today's Cash Inflows - Today's Cash Outflows`
- **Discrepancy:** `Actual Counted Cash - Expected Cash` (Positive = Overage, Negative = Shortage).
- **Carry-over:** Closing cash carries over as the next day's opening cash automatically.
- **Backfill:** Missing days (days with no settlement) are auto-backfilled to maintain continuity.

8. Inventory Rules

- **Ledger:** Stock is tracked via an append-only `StockMovement` ledger.
- **Cache:** `Product.current_stock` is a derived/cached value updated strictly via triggers or movement signals.
- **Movement Types:** `PURCHASE_IN`, `SALE_OUT`, `SALE_RETURN_IN`, `ADJUSTMENT`, `DAMAGE_OUT`, `TRANSFER_IN`, `TRANSFER_OUT`.
- **Service Items:** If `track_inventory=False` (e.g., labor, digital goods), no stock deduction occurs on sale.

9. Warranty Rules

- **Calculation:** `expiry_date = start_date + period_months`
- **Status Indicators:**
  - `ACTIVE`: Current date is before expiry.
  - `EXPIRING_SOON`: Current date is within 30 days of expiry.
  - `EXPIRED`: Current date is past expiry.
- **Binding:** Warranties are bound to a specific `ProductUnit` (serial number) for serialized items, or the invoice/item for non-serialized.

10. Service Ticket Rules

- **State Machine:** `RECEIVED` → `DIAGNOSING` → `AWAITING_PARTS` → `IN_REPAIR` → `READY` → `DELIVERED` (or `CANCELLED`).
- **Billing Calculation:** `Customer Bill = Labor (service_charge) + Σ Parts_Sell_Price - Discount`
- **Inventory Integration:** Parts used with `from_stock=True` automatically deduct from shop inventory upon ticket completion.

STOCKWHISK SAAS PLATFORM

DOC-05

End-to-End User Journeys & Operational Flows

20 Detailed Business Workflows, Preconditions & Step-by-Step Sequences

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Process Specification	Product Managers, UI/UX, QA, Operators	v1.4.0 Production	✅ Verified & Approved

Complete User Flows

This document details every major user flow step-by-step, including preconditions, postconditions, expected results, and related APIs.

1. User Registration & Onboarding Flow

- **Preconditions:** User has a valid phone number.
- **Actor:** New User/Shop Owner
- **Steps:**
  1. Enter name, phone, email, shop name.
  2. Request OTP.
  3. Enter OTP and set password.
- **Expected Results:** Shop created, User created as Owner, JWT token returned.
- **Error Scenarios:** Invalid OTP, Phone already exists.
- **APIs:** `POST /api/auth/register`, `POST /api/auth/verify-otp`

2. Login & Authentication Flow

- **Preconditions:** Account exists and is verified.
- **Actor:** Any User
- **Steps:**
  1. Enter phone and password.
  2. Click login.
- **Expected Results:** JWT access and refresh tokens returned, User profile loaded.
- **Error Scenarios:** Invalid credentials, Inactive account.
- **APIs:** `POST /api/auth/token`

3. POS Checkout Flow

- **Preconditions:** Cash drawer open, products in stock.
- **Actor:** Cashier
- **Steps:**
  1. Scan barcode or search product.
  2. Add to cart, adjust qty/price.
  3. Select/add customer.
  4. Select payment method(s) and enter amounts.
  5. Submit sale.
- **Expected Results:** Sale created, stock deducted, ledger updated, invoice printed.
- **Error Scenarios:** Insufficient stock, Invalid payment amount.
- **APIs:** `GET /api/pos/products`, `POST /api/pos/checkout`

4. Product Creation Flow

- **Preconditions:** User has product management permission.
- **Actor:** Manager/Admin
- **Steps:**
  1. Navigate to Products > Add New.
  2. Enter name, category, brand, cost, price.
  3. Toggle `is_serialized` or `track_inventory`.
  4. Save.
- **Expected Results:** Product record created.
- **Error Scenarios:** Duplicate SKU.
- **APIs:** `POST /api/inventory/products`

5. Product Purchase/Stock-In Flow

- **Preconditions:** Supplier exists.
- **Actor:** Manager/Admin
- **Steps:**
  1. Select Supplier.
  2. Add products, define cost and qty.
  3. If serialized, enter/scan serial numbers.
  4. Enter payment amount (full, partial, or credit).
  5. Save Purchase.
- **Expected Results:** Stock increased, Supplier payable updated, Ledger updated.
- **Error Scenarios:** Missing serials for serialized item.
- **APIs:** `POST /api/inventory/purchases`

6. Sales Return & Refund Flow

- **Preconditions:** Original sale exists.
- **Actor:** Cashier/Manager
- **Steps:**
  1. Lookup invoice.
  2. Select items to return and condition (restock or damage).
  3. Process refund amount.
- **Expected Results:** Stock adjusted based on condition, cash deducted, return invoice generated.
- **Error Scenarios:** Return qty exceeds sale qty.
- **APIs:** `POST` `/api/pos/returns`

7. Product Exchange/Replacement Flow

- **Preconditions:** Customer returning defective unit within warranty.
- **Actor:** Manager
- **Steps:**
  1. Lookup serial/invoice.
  2. Mark old unit as `TESTING_PENDING`.
  3. Issue new unit from stock.
  4. Transfer warranty.
- **Expected Results:** Inventory swapped, warranty updated.
- **Error Scenarios:** No replacement stock available.
- **APIs:** `POST` `/api/inventory/exchanges`

8. Due Collection Flow

- **Preconditions:** Customer has `due_balance > 0`.
- **Actor:** Cashier
- **Steps:**
  1. Select Customer.
  2. Enter collection amount and payment method.
  3. Apply to specific invoices or auto-apply to oldest.
  4. Save.
- **Expected Results:** Customer due reduced, cash increased, sale status updated.
- **Error Scenarios:** Collection exceeds due.
- **APIs:** `POST` `/api/customers/collect-due`

9. EMI Creation & Payment Flow

- **Preconditions:** Customer approved for EMI.
- **Actor:** Manager
- **Steps:**

1. At POS, select EMI payment.
2. Set down payment, interest rate, months.
3. Generate schedule.
4. Accept down payment.

- **Expected Results:** EMI schedule created, initial payment recorded.
- **Error Scenarios:** Invalid math configuration.
- **APIs:** `POST` `/api/pos/checkout` (with EMI payload), `POST` `/api/emi/pay-installment`

10. Daily Settlement/Shift Closing Flow

- **Preconditions:** End of day/shift.
- **Actor:** Cashier/Manager
- **Steps:**
  1. Open Settlement screen.
  2. Count physical cash.
  3. Enter actual cash amount.
  4. Submit.
- **Expected Results:** Expected vs Actual calculated, discrepancy logged, shift closed.
- **Error Scenarios:** Pending unhandled transactions.
- **APIs:** `POST` `/api/accounting/settlements`

11. Expense Recording Flow

- **Preconditions:** Expense categories configured.
- **Actor:** Any staff (with permission)
- **Steps:**
  1. Select Category.
  2. Enter amount, note, and payment source.
  3. Save.
- **Expected Results:** Ledger cash reduced, Expense recorded.
- **Error Scenarios:** Insufficient funds in selected account.
- **APIs:** `POST` `/api/accounting/expenses`

12. Capital Investment & Drawing Flow

- **Preconditions:** Owner account.
- **Actor:** Owner
- **Steps:**
  1. Go to Equity.
  2. Select Invest or Draw.
  3. Enter amount and account.
- **Expected Results:** Ledger and Equity totals updated.

- **APIs:** `POST /api/accounting/equity`

13. Account Transfer Flow

- **Preconditions:** Multiple accounts exist (e.g., Cash, Bank).
- **Actor:** Manager/Owner
- **Steps:**
  1. Select From Account and To Account.
  2. Enter amount.
  3. Save.
- **Expected Results:** Double ledger entry created, no net P&L change.
- **Error Scenarios:** Transfer exceeds available balance.
- **APIs:** `POST /api/accounting/transfers`

14. Warranty Verification Flow

- **Preconditions:** None.
- **Actor:** Staff
- **Steps:**
  1. Scan/enter serial number or invoice ID.
  2. View status.
- **Expected Results:** Displays Active/Expired status and days remaining.
- **APIs:** `GET /api/inventory/warranty-check`

15. Service Ticket Lifecycle Flow

- **Preconditions:** Device received for repair.
- **Actor:** Technician/Manager
- **Steps:**
  1. Create ticket (RECEIVED).
  2. Update to DIAGNOSING.
  3. Add parts and labor cost (IN_REPAIR).
  4. Mark READY and notify customer.
  5. Take payment and mark DELIVERED.
- **Expected Results:** Ticket tracks history, parts deducted upon delivery, cash collected.
- **APIs:** `POST /api/service/tickets`, `PATCH /api/service/tickets/{id}/status`

16. Barcode Generation & Printing Flow

- **Preconditions:** Products exist.
- **Actor:** Manager
- **Steps:**
  1. Select products and quantities.
  2. Configure label size.

3. Print via browser print dialog.

- **Expected Results:** Labels sent to printer.
- **APIs:** Frontend specific.

17. Item/Serial Lookup Flow

- **Preconditions:** None.
- **Actor:** Staff
- **Steps:**
  1. Global search serial number.
  2. View product lifecycle (purchase, sale, return history).
- **Expected Results:** Complete audit trail of unit.
- **APIs:** `GET` `/api/inventory/serials/{serial}`

18. Staff User Creation & Permission Assignment Flow

- **Preconditions:** Owner/Admin access.
- **Actor:** Owner
- **Steps:**
  1. Create user (name, phone, pin).
  2. Select Role (Cashier, Manager).
  3. Customize specific permissions if needed.
- **Expected Results:** Staff can log in with scoped access.
- **APIs:** `POST` `/api/users/staff`

19. Customer Profile & History Flow

- **Preconditions:** Customer exists.
- **Actor:** Any Staff
- **Steps:**
  1. Open Customer profile.
  2. View stats, due balance, and past invoices.
- **Expected Results:** Aggregated customer data displayed.
- **APIs:** `GET` `/api/customers/{id}`

20. Report Generation & Export Flow

- **Preconditions:** Data exists.
- **Actor:** Manager/Owner
- **Steps:**
  1. Navigate to Reports.
  2. Select date range and report type (Sales, P&L, Tax).
  3. Click Export to Excel/PDF.

- **Expected Results:** File downloaded.
- **APIs:** `GET` `/api/reports/sales?export=csv`

STOCKWHISK SAAS PLATFORM

DOC-06

Multi-Tenant Data Isolation & Security Framework

Thread-Local Isolation Contexts, Scoped ORM Guards & Defense-in-Depth

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Security Specification	Security Auditors, Backend Engineers	v1.4.0 Production	✅ Verified & Approved

Multi-Tenant Security Requirements

StockWhisk is a multi-tenant SaaS application. Strict data isolation between shops (tenants) is the highest security priority.

1. Architecture

- **Thread-Local Context:** The application uses a thread-local storage mechanism (`core.tenant_context`) to store the current active tenant for the duration of a request.
- **TenantMiddleware:** Extracts the `Shop` context from the authenticated JWT user on every request and sets it in the thread-local context.
- **TenantScopedModel:** An abstract base Django model that includes a `shop` ForeignKey. Almost all business data models inherit from this.
- **TenantManager:** A custom ORM manager applied to `TenantScopedModel` s that automatically filters ALL queries by the current tenant. If no tenant is set in the context, it returns `qs.none()` to prevent accidental data leaks.

2. Data Isolation Requirements

- **SEC-01:** Every database query MUST be scoped to the current tenant. The `TenantManager` handles this automatically, but raw queries must be strictly reviewed.
- **SEC-02:** No cross-tenant data access is allowed, even if a user manipulates IDs (e.g., changing an invoice ID in a URL to one belonging to another shop). The `TenantManager` prevents this by enforcing the `shop_id` filter.
- **SEC-03:** File uploads (shop logos, document backups) MUST be stored in tenant-scoped directories (e.g., `/media/shop_{id}/`).
- **SEC-04:** API responses MUST NEVER include relational data from other tenants. Serializers must use tenant-scoped querysets for relational fields.
- **SEC-05:** Bulk operations (creates, updates, deletes) MUST respect tenant boundaries and inject the correct `shop_id` .

3. Authentication & Authorization

- **Authentication:** JWT-based authentication with short-lived access tokens and longer-lived refresh tokens.
- **Authorization:** Role-Based Access Control (RBAC) per shop. A user's permissions are scoped strictly to their assigned role within a specific shop.
- **Platform Admin:** Platform administrators can impersonate tenants for support purposes. This MUST generate a severe audit log entry.
- **Demo Mode:** Read-only enforcement at the middleware/database level for demo accounts.
- **OTP Verification:** Phone number verification via OTP is required for registration. Password resets also utilize OTP.

4. API Security

- **Rate Limiting:** Enforced via DRF throttles (`BurstUserThrottle` , `UserRateThrottle`) to prevent brute force and DDoS attacks.
- **CORS:** Strictly configured to allow only trusted frontend domains.
- **HTTPS:** Enforced for all traffic; cookies are marked `Secure` and `HttpOnly` .
- **Input Validation:** Strict validation through DRF serializers. No direct mapping of request data to model fields without validation.
- **Injection Prevention:** Relying on Django ORM to prevent SQL injection. XSS prevention handled by React/Next.js escaping on the frontend and data sanitization on the backend if HTML is accepted.

5. Audit & Compliance

- **Financial Audit:** Append-only `LedgerEntry` system for all financial transactions. Modifications require reversing entries.
- **Inventory Audit:** Append-only `StockMovement` ledger.
- **Service Tracking:** `ServiceTicketStatusHistory` tracks every state change of a repair ticket.
- **User Tracking:** `last_seen` and IP tracking for active sessions.

6. Development Rules

- NEVER override `objects = models.Manager()` on a model that inherits from `TenantScopedModel` without explicitly re-implementing tenant filtering.
- ALWAYS use `TenantScopedModel` for new data models unless they are globally shared (e.g., Subscription Plans).

STOCKWHISK SAAS PLATFORM

DOC-07

Test Environment & Infrastructure Specification

Dockerized Staging, Hardware Scanners, Seed Data & CI/CD Pipelines

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
DevOps & QA	QA Engineers, DevOps, Infrastructure	v1.4.0 Production	✅ Verified & Approved

Test Environment Specification

This document details the test environment setup for the StockWhisk SaaS project, including the tech stack, environment types, required tools, and network access points.

Tech Stack

The StockWhisk platform is built using a modern, robust tech stack designed for performance, scalability, and maintainability.

- **Backend:** Python 3.11+, Django 4.2+, Django REST Framework 3.14+
- **Frontend:** Node.js 18+, Next.js 14.2, React 18
- **Database:** PostgreSQL 15+ (Production/Staging), SQLite (Local Development)
- **Cache/Queue:** Redis 7+ (used as Celery broker and caching layer)
- **Deployment:** Docker Compose managing the following services:
  - `backend` (Django application)
  - `frontend` (Next.js application)
  - `worker` (Celery background task worker)
  - `beat` (Celery Beat for scheduled tasks)
  - `caddy` (Reverse proxy for SSL and routing)
  - `postgres` (Database)
  - `redis` (Cache/Broker)
- **CI/CD:** GitHub Actions (for automated testing and deployment)

Environment Types

The system supports multiple environment types to facilitate development, testing, and production deployment.

1. Local Development

- **Purpose:** Rapid iteration and local debugging for developers.

- **Setup:** Uses SQLite for the database, Django development server (`runserver`), and Next.js development server.
- **Characteristics:** Hot-reloading enabled, simplified setup, minimal dependencies.

2. Docker Development

- **Purpose:** Local testing in an environment that closely mirrors production.
- **Setup:** Uses `docker-compose up` to orchestrate all services including PostgreSQL and Redis.
- **Characteristics:** Containerized isolation, consistent environment across developer machines.

3. Staging

- **Purpose:** Pre-production testing, QA, and client previews.
- **Setup:** Production-like Docker deployment on a staging VPS.
- **Characteristics:** Uses production database configuration, representative data, accessible via a staging subdomain.

4. Production

- **Purpose:** Live environment for end-users.
- **Setup:** Docker Compose on an Ubuntu VPS, utilizing Caddy for automatic SSL/HTTPS provisioning.
- **Characteristics:** HIGH availability, secure, optimized for performance.

Required Tools

To effectively test and develop on the StockWhisk platform, the following tools are required:

Software Dependencies

- Python 3.11+
- `pip` (Python package installer)
- `virtualenv` (or similar virtual environment manager)
- Node.js 18+
- `npm` (Node package manager)
- Docker & Docker Compose
- Git

Hardware/Peripheral Testing Tools

- **Barcode Scanner:** USB or Bluetooth barcode scanner for testing POS input and inventory management.
- **Thermal Printer:** 80mm ESC/POS compatible thermal printer for testing receipt generation and printing layouts.

Environment Setup Steps

Backend Setup (Local Development)

1. Create and activate a virtual environment: `python -m venv venv && source venv/bin/activate`
2. Install dependencies: `pip install -r requirements.txt`
3. Apply database migrations: `python manage.py migrate`

4. Create a superuser account: `python manage.py createsuperuser`
5. Run the development server: `python manage.py runserver`

Frontend Setup (Local Development)

1. Navigate to the frontend directory.
2. Install dependencies: `npm install`
3. Run the development server: `npm run dev`

Docker Setup (Development/Staging/Production)

1. Ensure Docker and Docker Compose are installed and running.
2. Build and start the containers in detached mode: `docker compose up -d --build`

Network & Access Points

When running the environment locally (either via dev servers or Docker), the services map to the following endpoints:

- **Backend API Base URL:** `http://localhost:8000/api/`
- **Frontend Application:** `http://localhost:3000`
- **Django Admin Panel:** `http://localhost:8000/admin/`

Note: In staging or production environments, replace `localhost` and the respective ports with the configured domain names.

STOCKWHISK SAAS PLATFORM

DOC-08

Test Account & Seed Data Specification

Multi-Shop Scenarios, Serialized Inventory Fixtures & Persona Sets

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
QA Specification	QA Engineers, Automation Teams	v1.4.0 Production	✅ Verified & Approved

Test Account and Data Specification

This document outlines the required test accounts and the essential test data set needed to comprehensively validate the StockWhisk platform, particularly focusing on multi-tenancy, role-based access control, and complex business logic.

Test Accounts

The following test accounts should be provisioned to verify different roles and permissions across the system. Ensure all accounts have a known, secure test password.

1. **Platform Super Admin**
   - **Email:** `admin@stockwhisk.com`
   - **Role:** Full platform access, Django admin access.
   - **Purpose:** System-wide configuration, tenant management.
2. **Shop Owner (Test Shop 1)**
   - **Email:** `owner@testshop1.com`
   - **Role:** Full access to Test Shop 1.
   - **Purpose:** Managing shop settings, viewing all reports, managing staff.
3. **Shop Manager (Test Shop 1)**
   - **Email:** `manager@testshop1.com`
   - **Role:** Configurable permissions within Test Shop 1 (e.g., inventory management, reporting, but no shop settings).
   - **Purpose:** Testing RBAC and limited administrative capabilities.
4. **Cashier (Test Shop 1)**
   - **Email:** `cashier@testshop1.com`
   - **Role:** POS-only access, daily shift management.
   - **Purpose:** Testing everyday sales workflows and preventing unauthorized access to backend modules.

5. **Shop Owner (Test Shop 2)**
   - **Email:** `owner@testshop2.com`
   - **Role:** Full access to Test Shop 2.
   - **Purpose:** Crucial for testing cross-tenant data isolation (ensuring Shop 1 cannot see Shop 2's data).
6. **Demo Shop User**
   - **Email:** `demo@stockwhisk.com` (or similar)
   - **Role:** Read-only access to a specific demo tenant.
   - **Purpose:** Safe environment for potential clients to explore the platform without altering data.
7. **Reseller**
   - **Email:** `reseller@test.com`
   - **Role:** Access to the Reseller portal.
   - **Purpose:** Testing tenant creation and management by authorized resellers.

Test Data Requirements

To effectively simulate real-world usage and test all features, the database must contain the following minimum data set:

Core Entities

- **Tenants (Shops):** At least 2 distinct active shops (to verify data isolation).
- **Categories:** 5+ product categories (e.g., Electronics, Accessories, Services).
- **Brands:** 5+ distinct brands.

Catalog & Inventory

- **Products:** 50+ diverse products distributed across categories/brands.
  - *Standard Inventory:* Items with `track_inventory=True`.
  - *Services/Non-inventory:* Items with `track_inventory=False` (e.g., Labor charge, generic accessory).
  - *Serialized Items:* Products utilizing `ProductUnit` models for individual serial number tracking.
- **Stock Levels:**
  - Items with 0 stock (Out of stock testing).
  - Items with low stock (<5 units, for reorder alert testing).
  - Items with normal/abundant stock.

CRM (Customers & Suppliers)

- **Customers:** 10+ customers.
  - Mix of customers with zero balance, positive due balances, and advanced payments.
- **Suppliers:** 5+ suppliers.
  - Mix of suppliers with zero payables and outstanding payables.

Transactions

- **Sales Invoices:** 20+ invoices.
  - Status mix: `PAID`, `PARTIAL` (with partial payments recorded), `DUE`.
  - Include various payment methods (Cash, Card, Bank Transfer).
- **Purchase Orders (POs):** 5+ POs.
  - Status mix: `RECEIVED` (fully added to inventory), `PENDING` (awaiting receipt).
- **EMI/Installments:** Active EMI schedules with recorded and pending installments.

Service & Maintenance

- **Service Tickets:** Tickets in various lifecycle stages (e.g., `RECEIVED`, `IN_PROGRESS`, `REPAIRED`, `DELIVERED`).
- **Warranties:**
  - Active warranties.
  - Warranties expiring soon (within 30 days).
  - Expired warranties.

Accounting & Financials

- **Expenses:** Expense records spanning various expense categories (e.g., Rent, Utilities, Salary).
- **Shifts/Settlements:**
  - Closed daily settlements (historical shift data).
  - An open active shift (for ongoing POS testing).
- **Ledger:** `LedgerEntry` records correctly reflecting all transactions across all account types (Assets, Liabilities, Equity, Revenue, Expenses).

Seed Data Strategy

To ensure consistency in testing environments, seed data should be managed programmatically.

Implementation Plan:
1. **Management Commands:** Develop custom Django management commands (e.g., `python manage.py seed_test_data`) to generate the required data programmatically using factories (like FactoryBoy) or predefined datasets.
2. **Fixtures:** Alternatively, maintain JSON fixtures for core, immutable data (e.g., default permissions, standard categories).
3. **Documentation:** The specific command or script to load this seed data must be documented in the main repository's README for developer onboarding.

STOCKWHISK SAAS PLATFORM

DOC-09

REST API Testing & Verification Specification

Endpoint Contracts, Request/Response Payloads, Status Codes & Auth

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
API Specification	API Testers, Backend Engineers, Integrators	v1.4.0 Production	✅ Verified & Approved

API Testing Specification

This document provides a comprehensive testing specification for the core API endpoints of the StockWhisk platform. It details expected behaviors, required payloads, and validation criteria, specifically emphasizing tenant data isolation and error handling.

General Testing Criteria for All Endpoints

For every endpoint tested, the following validations must be performed:
* **Authentication:** Verify that missing or invalid JWT tokens return a `401 Unauthorized`.
* **Tenant Isolation:** Verify that a user from Shop A cannot access, modify, or delete resources belonging to Shop B (returns `404 Not Found` or `403 Forbidden`).
* **Data Validation:** Verify that invalid request bodies (missing required fields, incorrect data types) return a `400 Bad Request` with descriptive error messages.
* **Method Allowed:** Verify that unsupported HTTP methods return a `405 Method Not Allowed`.

Authentication Endpoints

1. Initiate Registration

- **Method & URL:** `POST` `/api/accounts/register/initiate/`
- **Request Body:** `{"email": "newuser@test.com", "phone": "1234567890", ...}`
- **Expected Response:** `200 OK` (OTP sent)
- **Auth:** None
- **Errors:** `400` (Email/phone already exists)

2. Verify Registration

- **Method & URL:** `POST` `/api/accounts/register/verify/`
- **Request Body:** `{"email": "newuser@test.com", "otp": "123456"}`
- **Expected Response:** `201 Created` (Account created)

- **Auth:** None
- **Errors:** `400` (Invalid/Expired OTP)

3. Login

- **Method & URL:** `POST` `/api/accounts/login/`
- **Request Body:** `{"email": "user@test.com", "password": "password123"}`
- **Expected Response:** `200 OK` (Returns `access` and `refresh` JWTs)
- **Auth:** None
- **Errors:** `401` (Invalid credentials)

4. Refresh Token

- **Method & URL:** `POST` `/api/accounts/token/refresh/`
- **Request Body:** `{"refresh": "<refresh_token>"}`
- **Expected Response:** `200 OK` (Returns new `access` token)
- **Auth:** None
- **Errors:** `401` (Invalid/Expired refresh token)

5. Password Reset

- **Method & URL:** `POST` `/api/accounts/password-reset/`
- **Request Body:** `{"email": "user@test.com"}`
- **Expected Response:** `200 OK` (Reset link/OTP sent)
- **Auth:** None

Catalog Endpoints

1. Product Management

- **Method & URL:** `GET` `/api/catalog/products/` , `POST` `/api/catalog/products/`
- **Request Body (POST):** `{"name": "Item A", "price": 100, "category_id": 1, ...}`
- **Expected Response:** `200 OK` (List), `201 Created` (Details)
- **Auth:** Required (JWT)

2. Product Detail (CRUD)

- **Method & URL:** `GET` `/api/catalog/products/{id}/` , `PUT ...` , `DELETE ...`
- **Expected Response:** `200 OK` , `204 No Content` (DELETE)
- **Auth:** Required (JWT)
- **Errors:** `404` (Not found / Wrong tenant)

3. POS Product Grid (Optimized)

- **Method & URL:** `GET` `/api/catalog/products/?in_stock=1&light=1`
- **Expected Response:** `200 OK` (Optimized payload for POS UI)

- **Auth:** Required (JWT)

4. Product Units (Serialized Items)

- **Method & URL:** `GET` `/api/catalog/product-units/` , `POST ...`
- **Request Body (POST):** `{"product_id": 1, "serial_number": "SN12345"}`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

5. Categories & Brands

- **Method & URL:** `GET` `/api/catalog/categories/` , `POST ...` | `GET` `/api/catalog/brands/` , `POST ...`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

POS / Sales Endpoints

1. Checkout (Create Sale)

- **Method & URL:** `POST` `/api/pos/checkout/`
- **Request Body:** `{"items": [{"product_id": 1, "qty": 2}], "payment": {"amount": 200, "method": "CASH"}}`
- **Expected Response:** `201 Created` (Invoice details generated)
- **Auth:** Required (JWT)
- **Errors:** `400` (Insufficient stock, invalid payment amount)

2. Sales Invoices

- **Method & URL:** `GET` `/api/sales/sales/` (List), `GET` `/api/sales/sales/{id}/` (Detail)
- **Expected Response:** `200 OK`
- **Auth:** Required (JWT)

3. Add Payment to Invoice

- **Method & URL:** `POST` `/api/sales/sales/{id}/add_payment/`
- **Request Body:** `{"amount": 50, "method": "CARD"}`
- **Expected Response:** `200 OK` (Updated invoice status)
- **Auth:** Required (JWT)
- **Errors:** `400` (Payment exceeds due amount)

4. Correct Invoice

- **Method & URL:** `POST` `/api/sales/sales/{id}/correct/`
- **Request Body:** `{"reason": "Wrong item selected"}`
- **Expected Response:** `200 OK` (Status updated, inventory reversed)
- **Auth:** Required (Admin/Manager role)

5. Process Return

- **Method & URL:** `POST` `/api/sales/returns/`
- **Request Body:** `{"invoice_id": 1, "items": [{"product_id": 1, "qty_returned": 1}]}`
- **Expected Response:** `201 Created` (Return note created, inventory adjusted)
- **Auth:** Required (JWT)

Purchasing Endpoints

1. Purchase Orders

- **Method & URL:** `GET` `/api/purchasing/purchase-orders/`, `POST ...`
- **Request Body (POST):** `{"supplier_id": 1, "items": [{"product_id": 1, "qty": 10, "cost": 50}]}`
- **Expected Response:** `200 OK`, `201 Created`
- **Auth:** Required (JWT)

2. Receive Purchase Order

- **Method & URL:** `POST` `/api/purchasing/purchase-orders/{id}/receive/`
- **Request Body:** (Optional) Partial receipt details.
- **Expected Response:** `200 OK` (Inventory incremented, status updated)
- **Auth:** Required (JWT)
- **Errors:** `400` (Already received)

3. Suppliers

- **Method & URL:** `GET` `/api/purchasing/suppliers/`, `POST ...`
- **Expected Response:** `200 OK`, `201 Created`
- **Auth:** Required (JWT)

CRM Endpoints

1. Customers

- **Method & URL:** `GET` `/api/crm/customers/`, `POST ...`
- **Expected Response:** `200 OK`, `201 Created`
- **Auth:** Required (JWT)

2. Collect Due Payment

- **Method & URL:** `POST` `/api/crm/customers/{id}/pay-due/`
- **Request Body:** `{"amount": 100, "method": "BANK_TRANSFER"}`
- **Expected Response:** `200 OK` (Customer balance updated, ledger entry created)
- **Auth:** Required (JWT)

Accounting Endpoints

1. Expenses

- **Method & URL:** `GET /api/accounting/expenses/` , `POST ...`
- **Request Body (POST):** `{"category": "RENT", "amount": 1000}`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

2. Daily Settlements (Shifts)

- **Open Shift:** `POST /api/accounting/daily-settlements/open/`
  - **Body:** `{"opening_balance": 100}`
- **Close Shift:** `POST /api/accounting/daily-settlements/close/`
  - **Body:** `{"closing_balance_declared": 550}`
- **Expected Response:** `201 Created` , `200 OK`
- **Auth:** Required (JWT)

3. Financial Reports

- **Balance Sheet:** `GET /api/accounting/financial-position/`
- **P&L Report:** `GET /api/accounting/profit-report/`
- **Expected Response:** `200 OK` (Structured financial data)
- **Auth:** Required (Manager/Owner role)

Service Endpoints

1. Service Tickets

- **Method & URL:** `GET /api/service/tickets/` , `POST ...`
- **Request Body (POST):** `{"customer_id": 1, "device_info": "iPhone 12", "issue": "Broken screen"}`
- **Expected Response:** `200 OK` , `201 Created`
- **Auth:** Required (JWT)

2. Ticket Status Transition

- **Method & URL:** `POST /api/service/tickets/{id}/change_status/`
- **Request Body:** `{"new_status": "IN_PROGRESS"}`
- **Expected Response:** `200 OK`
- **Auth:** Required (JWT)

3. Add Repair Part

- **Method & URL:** `POST /api/service/tickets/{id}/add_part/`
- **Request Body:** `{"product_id": 5, "qty": 1}`

- **Expected Response:** `200 OK` (Cost added to ticket, inventory adjusted)
- **Auth:** Required (JWT)

4. Warranties

- **Method & URL:** `GET /api/service/warranties/`
- **Expected Response:** `200 OK` (List of active/expired warranties)
- **Auth:** Required (JWT)

STOCKWHISK SAAS PLATFORM

DOC-10

Comprehensive Functional Test Scenarios

60+ Prioritized Test Cases Covering POS, Accounting, Serial Units & Dues

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
QA Test Suite	QA Engineers, Release Managers	v1.4.0 Production	✅ Verified & Approved

Functional Test Scenarios

This document outlines detailed functional test scenarios for the StockWhisk SaaS project.

POS & Checkout (FT-POS-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-POS-001	Barcode scan adds product	Product with barcode exists in inventory	1. Open POS 2. Scan barcode	Product is added to cart with qty 1	P1
FT-POS-002	Serial unit selection	Serialized product is added to cart	1. Add serialized product 2. Open serial selector 3. Select a specific unit	Selected serial number is attached to the line item	P1
FT-POS-003	Zero-stock blocked	Tracked product has 0 stock	1. Attempt to add product via scan or search	System displays error; product not added to cart	P1
FT-POS-004	Cart quantity adjustment	Product is in cart	1. Click '+' to increase qty 2. Click '-' to decrease qty	Quantity updates correctly; price recalculates	P1
FT-POS-005	Multi-payment checkout	Cart has items total $100	1. Proceed to checkout 2. Enter $50 Cash and $50 Card 3. Complete sale	Invoice is created; payments are split correctly	P1
FT-POS-006	Quotation mode	Cart has items	1. Select 'Save as Quotation' 2. Complete	Quotation is saved; stock is NOT deducted	P2
FT-POS-007	EMI checkout	Customer selected, cart has items	1. Select EMI checkout 2. Enter down payment and months 3. Complete	Invoice created; EMI schedule generated	P1
FT-POS-008	Walk-in customer checkout	No customer selected	1. Add items 2. Checkout	Sale attributed to 'Walk-in Customer'	P1
FT-POS-009	Existing customer selection	Customer exists in DB	1. Search customer by name/phone 2. Select customer	Customer linked to current cart	P1
FT-POS-010	Discount application	Cart has items	1. Apply 10% discount 2. Verify total	Total reflects discount correctly	P2
FT-POS-011	Delivery charge	Cart has items	1. Add $10 delivery charge	Total increases by $10	P2

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-POS-012	VAT calculation	Product has 5% VAT	1. Add product to cart	VAT is calculated and displayed on total	P1

Products (FT-PRD-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-PRD-001	Create tracked product	Logged in as Admin	1. Go to Products -> Add 2. Fill details, enable tracking 3. Save	Product created; initial stock prompt appears	P1
FT-PRD-002	Create service item	Logged in as Admin	1. Add Product 2. Disable tracking, mark as Service 3. Save	Product created; no stock management applies	P1
FT-PRD-003	Edit product details	Product exists	1. Edit product price 2. Save	New price reflects in POS	P2
FT-PRD-004	Toggle track_inventory	Product has 0 stock	1. Edit product 2. Toggle tracking off	Tracking disabled; POS allows selling unlimited	P2
FT-PRD-005	Low stock badge	Product stock < alert level	1. View product list	Low stock badge is clearly visible	P3
FT-PRD-006	Generate barcode	Product exists without barcode	1. Select product 2. Click 'Generate Barcode'	Unique system barcode is generated and saved	P2
FT-PRD-007	Serial unit lifecycle	Serialized product exists	1. Receive serial X 2. Sell serial X 3. Return serial X	Status: In Stock -> Sold -> Returned/In Stock	P1

Inventory (FT-INV-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-INV-001	Stock increases on PO receive	Approved PO exists	1. Receive 10 units 2. Check inventory	Stock increased by 10	P1
FT-INV-002	Stock decreases on sale	Product has 10 stock	1. Sell 2 units via POS	Stock decreased to 8	P1
FT-INV-003	Stock returns on sale return	Sold invoice exists	1. Process return for 1 unit	Stock increased by 1	P1
FT-INV-004	Stock adjustment (Loss)	Product has 5 stock	1. Create adjustment: -1 unit (Damage) 2. Save	Stock becomes 4; reason logged	P2
FT-INV-005	Movement history accuracy	Multiple transactions	1. View product ledger	Chronological list of IN/OUT matches current stock	P1

Sales & Invoices (FT-SAL-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-SAL-001	Invoice listing	Sales exist	1. Go to Sales module	All invoices listed descending by date	P1
FT-SAL-002	Search by invoice#	Invoice INV-123 exists	1. Enter INV-123 in search	Only INV-123 is displayed	P2
FT-SAL-003	PDF generation	Invoice exists	1. Click Print/PDF icon	Proper A4/Thermal PDF is generated	P1
FT-SAL-004	Add payment to partial invoice	Invoice has $50 due	1. Open invoice 2. Add $20 payment	Due reduces to $30; Payment logged	P1
FT-SAL-005	Invoice correction	Invoice created today	1. Edit invoice notes 2. Save	Notes updated; financials unchanged	P3
FT-SAL-006	WhatsApp sharing	Invoice exists, customer has phone	1. Click Share -> WhatsApp	Opens WA with pre-filled message and link	P2

Returns (FT-RET-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-RET-001	Full return with cash refund	Paid invoice exists	1. Initiate full return 2. Select cash refund	Stock restored; Cash account reduced; Invoice marked Returned	P1
FT-RET-002	Partial return	Invoice has 2 items	1. Return 1 item	Invoice updated; partial stock restored	P1
FT-RET-003	Exchange/Replacement	Serialized item sold	1. Return Serial A 2. Issue Serial B	Stock swapped; financial diff is 0	P2
FT-RET-004	Refund to correct account	Invoice paid via Card	1. Return sale 2. Refund via Bank Transfer	Bank ledger reduced; Cash unchanged	P2
FT-RET-005	Serialized unit status revert	Serial X sold	1. Return Serial X	Serial X status = Available	P1

Customers & Dues (FT-CRM-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-CRM-001	Customer creation	Logged in	1. Add customer (Name, Phone) 2. Save	Customer created and searchable	P1
FT-CRM-002	Due balance accuracy	Customer has multiple credit sales	1. Check customer profile	Total due matches sum of unpaid invoices	P1
FT-CRM-003	Due collection updates balance	Customer owes $100	1. Collect $50 via Receive Payment	Due becomes $50; Cash increases	P1
FT-CRM-004	Due collection not counted as new sale	Collect due payment	1. Check Daily Sales Report	Sale total unchanged; Collection total increased	P1
FT-CRM-005	Customer payment history	Payments recorded	1. View customer ledger	All invoices and receipts listed chronologically	P2

EMI (FT-EMI-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-EMI-001	EMI schedule creation	EMI sale completed	1. View EMI details	Dates and amounts correctly distributed over X months	P1
FT-EMI-002	Installment payment	Installment is pending	1. Pay specific installment	Installment marked Paid; Next due date updated	P1
FT-EMI-003	Overdue tracking	Installment date passed	1. Check EMI dashboard	Installment highlighted in red as Overdue	P2

Accounting (FT-ACC-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-ACC-001	Expense entry	Cash available	1. Add $50 expense (e.g., Utility)	Cash reduced by $50; Expense account increased	P1
FT-ACC-002	Capital investment	Owner invests money	1. Record $1000 Capital	Cash increases; Equity increases	P2
FT-ACC-003	Drawing	Owner withdraws money	1. Record $100 Drawing	Cash decreases; Equity decreases; Not an expense	P2
FT-ACC-004	Account transfer	Multiple bank accounts	1. Transfer $100 Cash to Bank	Cash -100; Bank +100; Total liquid unchanged	P1
FT-ACC-005	P&L report accuracy	Sales and Expenses exist	1. Generate P&L	Gross Profit and Net Profit calculated correctly	P1

Service & Warranty (FT-SVC-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-SVC-001	Warranty verification	Serial number sold 1 month ago	1. Enter serial in Service module	System shows Warranty Active (if 1 year warranty)	P1
FT-SVC-002	Service ticket creation	Customer reports issue	1. Create ticket, assign technician	Ticket created in 'Open' state	P1
FT-SVC-003	Status transitions	Ticket is Open	1. Change to In Progress 2. Change to Resolved	Status updates correctly; timestamps logged	P2
FT-SVC-004	Part addition	Ticket requires part replacement	1. Add Part X to ticket	Part X stock deducted	P1
FT-SVC-005	Customer billing	Ticket resolved (Out of warranty)	1. Generate bill from ticket	Invoice created for parts + service charge	P2

Purchasing (FT-PUR-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
FT-PUR-001	PO creation	Supplier exists	1. Create PO for items 2. Save	PO generated in 'Pending' state	P1
FT-PUR-002	PO receive	Pending PO exists	1. Receive all items	Stock increases; PO state = 'Received'	P1
FT-PUR-003	Credit purchase	Receive PO without payment	1. Complete receive	Supplier payable increases by PO amount	P1
FT-PUR-004	Supplier payment	Supplier payable exists	1. Pay supplier $100	Payable reduces by $100; Cash reduces	P1

STOCKWHISK SAAS PLATFORM

DOC-11

Security & Multi-Tenant Penetration Test Scenarios

Threat Models, Cross-Tenant Leaks, RBAC Bypasses & Throttling Audits

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Security Test Suite	Security Engineers, Pentesting Teams	v1.4.0 Production	✅ Verified & Approved

Security Test Scenarios

This document outlines detailed security test scenarios for the StockWhisk SaaS project.

Authentication (ST-AUTH-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-AUTH-001	Login with valid credentials	User exists	1. Enter valid email/password 2. Submit	User logged in; JWT token returned	P1
ST-AUTH-002	Login with invalid credentials	User exists	1. Enter invalid password 2. Submit	Login failed; generic error message	P1
ST-AUTH-003	JWT token expiry	User is logged in	1. Wait for token expiration 2. Attempt API call	401 Unauthorized; User prompted to re-login or refresh	P1
ST-AUTH-004	Token refresh	Refresh token is valid	1. Call refresh endpoint	New access token issued	P1
ST-AUTH-005	Brute force lockout	User exists	1. Attempt invalid login 10 times	Account temporarily locked; 429/403 returned	P2
ST-AUTH-006	Password reset	User forgets password	1. Request reset link 2. Use link to set new password	Password updated securely; old tokens invalidated	P2

Authorization (ST-AUTHZ-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-AUTHZ-001	Cashier access control	Logged in as Cashier	1. Attempt to access Admin Settings via URL/API	403 Forbidden; UI hides option	P1
ST-AUTHZ-002	Manager user management	Logged in as Manager w/o user perms	1. Attempt to create new user	403 Forbidden	P1
ST-AUTHZ-003	Owner full access	Logged in as Owner	1. Access all modules	Full read/write access granted	P1
ST-AUTHZ-004	Permission boundary	User has only 'View Sales'	1. Attempt to 'Edit Sale' via API	403 Forbidden	P1

Tenant Isolation (ST-TENANT-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-TENANT-001	Cross-tenant product isolation	Shop A and Shop B exist	1. Login to Shop A 2. Query products	Only Shop A products returned	P1
ST-TENANT-002	Cross-tenant ID manipulation	Shop B invoice ID is 500	1. Login to Shop A 2. Request `/api/invoices/500/`	404 Not Found (or 403)	P1
ST-TENANT-003	API response integrity	Standard API call	1. Fetch sales list	Verify no `tenant_id` from other shops leaked	P1
ST-TENANT-004	Bulk operation boundaries	Upload CSV products	1. Upload generic CSV	Products strictly bound to uploader's tenant	P1
ST-TENANT-005	File upload isolation	Shop A uploads logo	1. Shop B attempts to access URL (if private)	Access denied / strict scoping applied	P2

Data Validation (ST-DATA-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-DATA-001	SQL injection prevention	Search input	1. Enter `' OR 1=1;--` in search	Input sanitized; no SQL execution	P1
ST-DATA-002	XSS prevention	Customer creation	1. Enter `<script>alert(1)</script>` as name	Script tags escaped/stripped on render	P1
ST-DATA-003	Negative quantity rejection	POS / API	1. Submit cart with qty -5	Validation error; 400 Bad Request	P1
ST-DATA-004	Price validation	POS / API	1. Submit sale with negative price	Validation error	P1
ST-DATA-005	Required field enforcement	Product creation	1. Submit without Name	400 Bad Request; clear error message	P2

Rate Limiting (ST-RATE-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-RATE-001	API throttling	Normal user	1. Send 1000 requests in 1 minute	429 Too Many Requests after threshold	P2
ST-RATE-002	Burst rate limiting	Normal user	1. Send 50 concurrent requests instantly	Regulated by burst allowance; excess rejected	P2

Demo Mode (ST-DEMO-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-DEMO-001	Write operation block	Tenant is Demo Shop	1. Attempt to create sale / edit product	Operation blocked; "Demo mode" message	P2
ST-DEMO-002	Read operation allow	Tenant is Demo Shop	1. View dashboard, reports	Data loads successfully	P2

Impersonation (ST-IMP-xxx)

TEST ID	TEST NAME	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
ST-IMP-001	Admin impersonation	Superadmin user	1. Click impersonate Shop A	Logged in context changes to Shop A	P1
ST-IMP-002	Impersonation audit trail	Admin impersonates	1. Perform action as Shop A	Action logged with Admin's original ID in audit	P2
ST-IMP-003	Non-admin restriction	Regular tenant user	1. Attempt impersonation API call	403 Forbidden	P1

STOCKWHISK SAAS PLATFORM

DOC-12

Performance, Load & Stress Benchmark Scenarios

Concurrency Benchmarks, High-Volume POS Latency & Query Profiling

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Performance Benchmark	Performance Engineers, Tech Leads	v1.4.0 Production	✅ Verified & Approved

Performance Test Scenarios

This document outlines detailed performance test scenarios and benchmarks for the StockWhisk SaaS project.

Load Testing

TEST ID	SCENARIO	TARGET METRIC	NOTES
PT-LOAD-001	50 concurrent POS checkout transactions	< 2s response time	Simulates peak hour retail checkout volume across multiple tenants.
PT-LOAD-002	100 concurrent product searches	< 500ms response time	Focuses on database read replica and search indexing performance.
PT-LOAD-003	Dashboard load with 10,000+ sales records	< 3s response time	Tests aggregation queries (SUM, COUNT) for daily/monthly metrics.
PT-LOAD-004	Report generation with 1 year of data	< 5s response time	Tests heavy background processing or optimized materialized views.
PT-LOAD-005	Barcode scan response	< 200ms response time	CRITICAL for POS UX; requires minimal latency.

Stress Testing

TEST ID	SCENARIO	EXPECTED BEHAVIOR UNDER STRESS
PT-STRESS-001	200 concurrent users on POS	Graceful degradation; no data corruption; latency increases but requests don't drop.
PT-STRESS-002	Bulk product import (1000+ items) via CSV	Background task queue handles load; UI remains responsive; import completes within 1 minute.
PT-STRESS-003	Database query spike	Connection pool handles requests without dropping; slow queries logged for review.

Scalability

TEST ID	SCENARIO	ARCHITECTURE FOCUS
PT-SCALE-001	System handles 100+ active shops	Multi-tenant routing efficiency; global cache hit rates.
PT-SCALE-002	Single shop with 50,000+ products	Indexing on `tenant_id` + `product_id`; pagination efficiency.
PT-SCALE-003	Single shop with 100,000+ invoices	Date-range query efficiency; storage cost optimization for PDF generation.

Response Time Benchmarks

The following targets must be maintained under standard load:

- **POS Barcode Scan:** < 200ms
- **Checkout Completion (Sale save):** < 2s
- **Dashboard Initial Load:** < 3s
- **Custom Report Generation:** < 5s
- **Product Search (Typeahead):** < 500ms
- **Invoice PDF Generation & Download:** < 3s

Database Performance Requirements

- **Index Verification:** Ensure B-Tree indexes exist on all foreign keys and frequently searched columns (e.g., `barcode`, `phone`, `invoice_number`).
- **N+1 Query Prevention:** Utilize Django's `select_related` and `prefetch_related` on all listing APIs to strictly cap the query count per request (target < 5 queries per complex list).
- **Connection Pooling:** Verify PgBouncer (or similar) is configured correctly to prevent connection exhaustion during peak load.

STOCKWHISK SAAS PLATFORM

DOC-13

Continuous Regression Testing & Release Plan

Critical Path Verification Suites, Rollback Gates & Release Checklists

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
QA Strategy	QA Leads, Release Engineers, DevOps	v1.4.0 Production	✅ Verified & Approved

Regression Testing Plan

Purpose

Ensure that new code changes do not break existing functionality across the StockWhisk multi-tenant SaaS application.

Regression Test Suite Organization

CRITICAL **Path Tests (Must pass before ANY release)**

1. User login and authentication
2. POS checkout (scan → cart → payment → invoice)
3. Product creation and stock tracking
4. Sales return and refund
5. Due collection
6. Daily settlement
7. Multi-tenant data isolation

Module Regression Tests

POS Module

- **Key Test Cases:** Add to cart by barcode, apply discount, split payment, generate invoice.
- **Data Dependencies:** Active products with stock, valid tax configurations.
- **Expected Execution Time:** 5 mins (Automated), 15 mins (Manual)

Products Module

- **Key Test Cases:** Create product with variants, update pricing, bulk import, barcode generation.
- **Data Dependencies:** Categories, Brands, Units defined.
- **Expected Execution Time:** 3 mins (Automated)

Inventory Module

- **Key Test Cases:** Stock transfer between branches, stock adjustment, low stock alerts.
- **Data Dependencies:** Multiple branches/warehouses, existing stock.
- **Expected Execution Time:** 4 mins (Automated)

Sales & Returns

- **Key Test Cases:** View sales history, process full return, process partial return, refund to customer balance.
- **Data Dependencies:** Completed sales records.
- **Expected Execution Time:** 5 mins (Automated)

CRM Module

- **Key Test Cases:** Add customer, update credit limit, customer ledger view.
- **Data Dependencies:** None.
- **Expected Execution Time:** 2 mins (Automated)

EMI Module

- **Key Test Cases:** Create EMI plan, calculate interest, process monthly installment, calculate late fees.
- **Data Dependencies:** EMI-enabled products, active customers.
- **Expected Execution Time:** 6 mins (Automated)

Accounting Module

- **Key Test Cases:** Chart of accounts tree, journal entry, trial balance generation, P&L report.
- **Data Dependencies:** Existing financial transactions.
- **Expected Execution Time:** 7 mins (Automated)

Service Module

- **Key Test Cases:** Create service job, assign technician, update job status, invoice service.
- **Data Dependencies:** Active technicians, service items.
- **Expected Execution Time:** 4 mins (Automated)

Purchasing Module

- **Key Test Cases:** Create PO, receive goods, supplier payment, purchase return.
- **Data Dependencies:** Active suppliers, product master data.
- **Expected Execution Time:** 5 mins (Automated)

Settings Module

- **Key Test Cases:** Update store profile, configure tax rates, role management, receipt template changes.
- **Data Dependencies:** Admin access.
- **Expected Execution Time:** 3 mins (Automated)

Integration Tests

- **POS → Stock deduction → Accounting ledger:** Verify that a sale correctly reduces inventory and logs revenue.
- **Purchase → Stock increase → Supplier payable:** Verify receiving a PO increases stock and updates supplier balance.

- **Return → Stock restock → Cash refund ledger:** Verify returns increase stock and debit the cash account.
- **Due collection → Cash increase → Customer balance update:** Verify receiving payment clears customer dues and credits cash.
- **Settlement → Expected cash calculation:** Verify end-of-day settlement correctly aggregates all cash inflows/outflows.

Execution Schedule

- Before every production deployment
- After database migrations
- After major feature additions
- Weekly scheduled runs (Automated)

Tools & Framework

- **Backend:** `pytest` + Django TestCase
- **Frontend:** Manual smoke testing checklist (Future: Playwright/Cypress)
- **API:** Postman collection or automated scripts

Rollback Criteria

- P1 **Failures:** Any P1 test failure blocks deployment immediately.
- P2 **Failures:** Require team review; generally blocks unless an acceptable workaround is approved.
- P3 **Failures:** Are logged in the issue tracker for resolution in the next sprint, deployment may proceed.

STOCKWHISK SAAS PLATFORM

DOC-14

Enterprise Defect Reporting Standard & Template

Standardized Bug Taxonomy, Severity Definitions & Real Production Examples

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
QA Standard	QA Engineers, Support Teams, Developers	v1.4.0 Production	✅ Verified & Approved

Bug Report Template

Bug Report Template Fields

- **Bug ID**: AUTO (e.g., BUG-2024-0001)
- **Title**: Clear, concise summary
- **Reporter**: Name & role
- **Date Reported**: YYYY-MM-DD
- **Environment**: Production / Staging / Local Dev
- **Browser/Device**: Chrome 120, Windows 11 / Safari, iPhone 15
- **Module**: POS / Products / Sales / Accounting / etc.
- **Severity**: CRITICAL / MAJOR / MINOR / Cosmetic
- **Priority**: P1 (Blocker) / P2 (HIGH) / P3 (MEDIUM) / P4 (Low)
- **Status**: New / Assigned / In Progress / Fixed / Verified / Closed / Reopened

Description Section

- **Summary**: One-line description
- **Steps to Reproduce**: Numbered step-by-step
- **Expected Result**: What should happen
- **Actual Result**: What actually happened
- **Screenshots/Videos**: Attachments (links or inline)
- **Console Errors**: Browser dev tools output
- **API Response**: If applicable, include status code and response body

Classification

Severity Definitions

- CRITICAL: System crash, data loss, security breach, financial calculation error.
- MAJOR: Feature completely broken, wrong calculations, data corruption.
- MINOR: Feature partially broken, UI inconsistency, workaround available.
- **Cosmetic**: Visual/text issues, typos, alignment.

Priority Definitions

- P1 **(Blocker)**: Must fix immediately (blocks business operations).
- P2 (HIGH): Fix in current sprint.
- P3 (MEDIUM): Fix in next sprint.
- **P4 (Low)**: Fix when convenient.

Example Bug Reports

Example 1: CRITICAL Financial Error

- **Bug ID**: BUG-2024-0042
- **Title**: POS calculates incorrect tax when discount is applied
- **Reporter**: QA Lead
- **Date Reported**: 2024-10-15
- **Environment**: Staging
- **Browser/Device**: Chrome 120, Windows 11
- **Module**: POS
- **Severity**: CRITICAL
- **Priority**: P1
- **Status**: New
- **Summary**: The tax amount is calculated on the pre-discount total instead of the post-discount total, leading to overcharging.
- **Steps to Reproduce**:
  1. Add item A ($100) to POS cart.
  2. Apply a 10% discount.
  3. Observe the calculated 10% tax.
- **Expected Result**: Tax should be $9 (10% of $90).
- **Actual Result**: Tax is $10 (10% of $100).
- **Screenshots**: [Link]
- **API Response**: `POST` `/api/pos/calculate` returned incorrect `tax_amount`.

Example 2: MAJOR Feature Failure

- **Bug ID**: BUG-2024-0043
- **Title**: Cannot upload product images in Product Creation
- **Reporter**: Content Manager
- **Date Reported**: 2024-10-16
- **Environment**: Production
- **Browser/Device**: Safari 17, macOS
- **Module**: Products
- **Severity**: MAJOR
- **Priority**: P2
- **Status**: Assigned
- **Summary**: Uploading a PNG image throws a 500 error.
- **Steps to Reproduce**:
  1. Navigate to Products -> Add Product.
  2. Fill required fields.
  3. Select a valid PNG image under 2MB.
  4. Click Save.
- **Expected Result**: Product saves with image successfully.
- **Actual Result**: Server error 500 displayed on UI.
- **Console Errors**: `Failed to load resource: the server responded with a status of 500`
- **API Response**: `{"error": "Unsupported Media Type"}`

Example 3: Cosmetic Issue

- **Bug ID**: BUG-2024-0044
- **Title**: Typo in "Settlement" button on Dashboard
- **Reporter**: UI/UX Designer
- **Date Reported**: 2024-10-17
- **Environment**: Local Dev
- **Browser/Device**: Firefox 115, Windows 10
- **Module**: Dashboard
- **Severity**: Cosmetic
- **Priority**: P4
- **Status**: New
- **Summary**: The button says "Setlement" instead of "Settlement".
- **Steps to Reproduce**:
  1. Login to admin dashboard.
  2. Look at the top right quick action bar.
- **Expected Result**: Button reads "Settlement".
- **Actual Result**: Button reads "Setlement".

STOCKWHISK SAAS PLATFORM

DOC-15

Final QA Acceptance & Release Sign-Off Gate

Production Deployment Criteria, Integrity Audits & Multi-Tier Sign-Offs

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Release Governance	Product Owners, QA Leads, CTO	v1.4.0 Production	✅ Verified & Approved

Final QA Acceptance Criteria

Release Readiness Checklist

1. Functional Acceptance

- [ ] All P1 functional test scenarios pass
- [ ] All P2 functional test scenarios pass (95%+ pass rate)
- [ ] POS checkout flow works end-to-end (scan → payment → invoice print)
- [ ] Financial calculations are accurate (cash flow, P&L, settlement)
- [ ] EMI calculations match specification
- [ ] Returns/refunds correctly update stock and cash
- [ ] Due collection correctly updates customer balance without duplicating revenue
- [ ] Multi-tenant isolation verified (Shop A cannot see Shop B data)
- [ ] All user roles have correct access permissions
- [ ] Bangla and English translations work correctly on all pages

2. Security Acceptance

- [ ] All security test scenarios pass
- [ ] No cross-tenant data leakage
- [ ] JWT authentication working correctly
- [ ] Rate limiting enforced
- [ ] Demo mode properly restricts write operations
- [ ] Input validation prevents injection attacks

3. Performance Acceptance

- [ ] POS barcode scan responds within 200ms
- [ ] Dashboard loads within 3 seconds
- [ ] Checkout completes within 2 seconds

- [ ] System handles 50+ concurrent POS sessions
- [ ] No N+1 query issues in critical paths

4. UI/UX Acceptance

- [ ] Responsive design works on mobile, tablet, desktop
- [ ] Dark mode and light mode both render correctly
- [ ] All popover/modal components display properly
- [ ] Print layouts (thermal 80mm and A4) format correctly
- [ ] Navigation and sidebar work correctly

5. Data Integrity Acceptance

- [ ] Product stock matches StockMovement ledger sum
- [ ] Customer due balance matches unpaid invoice totals
- [ ] Cash balance matches LedgerEntry calculations
- [ ] Supplier payable matches unpaid PO totals
- [ ] Settlement expected cash matches ledger-derived value

6. Deployment Acceptance

- [ ] Docker build succeeds without errors
- [ ] Database migrations run cleanly
- [ ] Static assets build and serve correctly
- [ ] Caddy SSL certificates valid
- [ ] Celery worker and beat running
- [ ] Backup system functional

Sign-Off

- **QA Lead Approval**: ___ ***Date:*** _
- **Product Owner Approval**: ___ ***Date:*** _
- **Technical Lead Approval**: ___ ***Date:*** _
- **Deployment Date and Version**: _____

STOCKWHISK SAAS PLATFORM

DOC-16

Database Schema & Entity-Relationship (ER) Specification

Complete 54-Model Multi-Tenant Relational Schema, Mermaid ERDs & Data Dictionary

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Database Architecture	Database Architects, Backend Engineers, DevOps	v1.4.0 Production	✅ Verified & Approved

16. StockWhisk Database Schema & Entity-Relationship (ER) Specification

1. Executive Summary & Architecture Overview

The StockWhisk relational database architecture is engineered as a **Multi-Tenant SaaS Retail ERP, POS, RMA Service, and Financial Accounting engine**. It is built on Django ORM with native support for PostgreSQL (Production), MySQL, and SQLite (Local Development).

Core Architectural Principles:

1. **Tenant Isolation by Construction**: All business entities inherit from `TenantScopedModel` (via `core.models.TenantScopedModel`), which automatically injects a `shop_id` foreign key and enforces tenant query filtering through `TenantManager`.
2. **Immutable Financial & Stock Ledgers (Event-Sourced)**:
   - Cash/Bank movements are logged into the append-only `LedgerEntry` table.
   - Physical warehouse movements are recorded in the append-only `StockMovement` table.
   - Derived quantities (`Product.current_stock`, `Customer.due_balance`, `Supplier.due_balance`) are cached projections protected by database constraints and atomic transactions.
3. **Dual-Layer Inventory Tracking**:
   - Standard bulk inventory tracked by quantity.
   - HIGH-value serialized inventory tracked per individual physical piece (`ProductUnit`) with unique barcodes and serialized lifecycle states.

2. HIGH-Level Entity-Relationship (ER) Diagrams

2.1 Core Multi-Tenancy, Auth & Subscriptions ERD

```
erDiagram
    SubscriptionPlan |├-o{ Shop : "subscribed_under"
    SubscriptionPlan |├-o{ Subscription : "defines_terms"
    Shop |├-o{ Subscription : "has_history"
    Shop |├-o{ Branch : "operates"
    Shop |├-o{ Role : "defines_roles"
    Shop |├-o{ User : "employs"
    Shop |├-o{ APIKey : "provisions"
    Branch |├-o{ User : "assigned_to"
    Role }|--|{ Permission : "grants"
    User |├-o{ Notification : "receives"
    Subscription |├-o{ SubscriptionInvoice : "bills"
    SubscriptionInvoice |├-o{ ManualPayment : "settled_by"

    Shop {
        int id PK
        string name
        string slug
        string subdomain
        string currency
        decimal vat_percent
        boolean emi_enabled
        boolean is_active
    }

    Branch {
        int id PK
        int shop_id FK
        string name
        string code
        boolean is_main
    }

    User {
        int id PK
        int shop_id FK
        int branch_id FK
        string email
        string role
        string phone
        datetime last_seen
    }

    SubscriptionPlan {
        int id PK
        string name
        string tier
        decimal price_monthly
        decimal price_yearly
        int max_users
        int max_branches
        int max_products
    }
```

2.2 Product Catalog, Purchasing & Stock Movements ERD

```
erDiagram
    Shop ||--o{ Category : "categorizes"
    Shop ||--o{ Brand : "brands"
    Shop ||--o{ Product : "owns"
    Category ||--o{ Product : "classifies"
    Brand ||--o{ Product : "brands"
    Supplier ||--o{ Product : "primary_supplier"
    Product ||--o{ ProductVariation : "has_variants"
    Product ||--o{ ProductUnit : "tracks_serials"
    Product ||--o{ StockMovement : "logs_movements"
    Branch ||--o{ StockMovement : "movement_location"
    Supplier ||--o{ PurchaseOrder : "receives_from"
    PurchaseOrder ||--o{ PurchaseOrderItem : "contains"
    Product ||--o{ PurchaseOrderItem : "ordered_item"
    PurchaseOrder ||--o{ PurchasePayment : "paid_via"
    Supplier ||--o{ SupplierPayment : "settles_due"

    Product {
        int id PK
        int shop_id FK
        string name
        string sku
        string barcode
        decimal cost_price
        decimal selling_price
        decimal current_stock
        boolean track_inventory
        int reorder_level
    }

    ProductUnit {
        int id PK
        int product_id FK
        string barcode UK
        string status
        decimal cost_price
        decimal selling_price
        int warranty_months
        int sale_id FK
    }

    StockMovement {
        int id PK
        int product_id FK
        int branch_id FK
        string movement_type
        decimal quantity
        decimal unit_cost
        string reference_type
        string reference_id
    }
```

2.3 POS Sales, Invoicing, Returns, CRM & EMI ERD

```
erDiagram
    Shop ||--o{ Customer : "manages"
    Customer ||--o{ Sale : "purchases"
    Customer ||--o{ CustomerPayment : "pays_due"
    Customer ||--o{ EMISchedule : "finances"
    Branch ||--o{ Sale : "originates"
    Sale ||--o{ SaleItem : "contains"
    Product ||--o{ SaleItem : "sold_product"
    Sale ||--o{ Payment : "settled_with"
    Sale ||--o{ SaleReturn : "returned_in"
    SaleReturn ||--o{ SaleReturnItem : "contains_items"
    Sale ||--o| EMISchedule : "installment_plan"
    EMISchedule ||--o{ EMIInstallment : "monthly_dues"

    Sale {
        int id PK
        int shop_id FK
        int customer_id FK
        string invoice_no UK
        datetime sale_date
        decimal subtotal
        decimal discount
        decimal delivery_charge
        decimal tax
        decimal total
        decimal paid
        string status
    }

    SaleItem {
        int id PK
        int sale_id FK
        int product_id FK
        decimal quantity
        decimal unit_price
        decimal unit_cost
        decimal discount
        decimal subtotal
    }

    EMISchedule {
        int id PK
        int sale_id FK
        int customer_id FK
        decimal total_emi_amount
        decimal down_payment
        decimal interest_percent
        int total_months
        decimal monthly_installment
        string status
    }

    EMIInstallment {
        int id PK
        int schedule_id FK
        int installment_number
        date due_date
        decimal amount
        decimal paid_amount
```

```
        string status
    }
```

2.4 Financial Accounting, Daily Settlement & Cash Flow ERD

```
erDiagram
    Shop ||--o{ ExpenseCategory : "categorizes"
    ExpenseCategory ||--o{ Expense : "classifies"
    Shop ||--o{ Expense : "incurs"
    Shop ||--o{ LedgerEntry : "records_cashflow"
    Shop ||--o{ DailySettlement : "reconciles"
    Shop ||--o{ Investment : "equity_movements"
    Shop ||--o{ AccountTransfer : "liquid_transfers"

    LedgerEntry {
        int id PK
        int shop_id FK
        string account
        decimal amount
        string source_type
        string source_id
        string description
        datetime created_at
    }

    DailySettlement {
        int id PK
        int shop_id FK
        datetime opened_at
        datetime closed_at
        decimal opening_cash
        decimal expected_cash
        decimal actual_cash
        decimal discrepancy
        decimal total_sales
        decimal total_expenses
        decimal total_refunds
        string status
    }

    Investment {
        int id PK
        int shop_id FK
        string investor_name
        string type
        decimal amount
        date invested_on
        string payment_method
    }

    AccountTransfer {
        int id PK
        int shop_id FK
        string from_account
        string to_account
        decimal amount
        date transferred_on
        string reference
    }
```

2.5 RMA Warranty & Repair Services ERD

```
erDiagram
    Product |├-o{ Warranty : "covered_by"
    ProductUnit |├-o{ Warranty : "bound_to"
    Customer |├-o{ Warranty : "registered_to"
    Warranty |├-o{ WarrantyClaim : "claims"
    Warranty |├-o{ ServiceTicket : "service_job"
    Customer |├-o{ ServiceTicket : "customer_device"
    User |├-o{ ServiceTicket : "assigned_technician"
    ServiceTicket |├-o{ ServiceTicketPart : "parts_consumed"
    Product |├-o{ ServiceTicketPart : "inventory_part"
    ServiceTicket |├-o{ ServiceTicketStatusHistory : "audit_log"

    Warranty {
        int id PK
        int product_id FK
        int product_unit_id FK
        int customer_id FK
        string serial_no
        int period_months
        date start_date
        date expiry_date
        string status
    }

    ServiceTicket {
        int id PK
        int shop_id FK
        int customer_id FK
        int warranty_id FK
        int technician_id FK
        string ticket_no UK
        string device_type
        string issue_type
        string status
        decimal service_charge
        decimal discount
        decimal paid
    }

    ServiceTicketPart {
        int id PK
        int ticket_id FK
        int product_id FK
        decimal quantity
        decimal unit_cost
        decimal unit_price
        boolean from_stock
    }
```

3. Complete Data Dictionary (Table by Table)

3.1 App: `tenants`

Table: `tenants_subscriptionplan`

Defines global SaaS billing tiers, feature quotas, and limits.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`name`	CharField(100)	No		Display name (e.g. Free, Basic, Pro, Enterprise)
`tier`	CharField(20)	No	Enum (`FREE` , `BASIC` , `PROFESSIONAL` , `ENTERPRISE`)	Plan tier identifier
`price_monthly`	DecimalField(10,2)	No	Default `0.00`	Monthly billing price
`price_yearly`	DecimalField(10,2)	No	Default `0.00`	Yearly billing price
`features`	JSONField	No	Default `list`	Active feature toggles
`max_users`	PositiveIntegerField	No	Default `1`	Max user limit
`max_branches`	PositiveIntegerField	No	Default `1`	Max branches allowed
`max_products`	PositiveIntegerField	No	Default `100`	Max product catalog size
`is_active`	BooleanField	No	Default `True`	Whether plan is purchasable

Table: `tenants_shop`

The root tenant partition representing an independent business.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`name`	CharField(150)	No		Legal / Commercial business name
`slug`	SlugField(160)	No	Unique	URL slug
`subdomain`	CharField(63)	Yes	Unique	Optional dedicated subdomain
`logo`	ImageField	Yes		Uploaded business brand logo
`phone`	CharField(20)	No		Primary contact phone
`email`	EmailField	Yes		Primary business notification email
`currency`	CharField(10)	No	Default `'BDT'`	Currency code (BDT, USD, etc.)
`vat_enabled`	BooleanField	No	Default `False`	Whether VAT is charged on sales
`vat_percent`	DecimalField(5,2)	No	Default `0.00`	Percentage rate of VAT
`emi_enabled`	BooleanField	No	Default `True`	Enable installment sales
`offline_sale_mode`	BooleanField	No	Default `False`	Enable offline cached POS sync
`plan_id`	ForeignKey	Yes	FK -> `tenants_subscriptionplan`	Current active subscription plan
`trial_ends_at`	DateTimeField	Yes		Free trial expiration timestamp
`is_active`	BooleanField	No	Default `True`	Active shop status
`is_demo`	BooleanField	No	Default `False`	Public read-only demo shop
`is_free`	BooleanField	No	Default `False`	Lifetime free granted shop

Table: `tenants_branch`

Physical outlet / warehouse branches under a single tenant shop.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Parent tenant shop
`name`	CharField(100)	No		Branch name (e.g. Main Branch, Gulshan Outlet)
`code`	CharField(20)	Yes		Unique branch short code
`phone`	CharField(20)	Yes		Branch telephone / mobile
`address`	TextField	Yes		Physical location address
`is_main`	BooleanField	No	Default `False`	Designates primary headquarters branch
`is_active`	BooleanField	No	Default `True`	Branch operational status

3.2 App: `accounts`

Table: `accounts_user`

System user accounts (Cashiers, Managers, Owners, Resellers, Platform Superadmins).

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`email`	EmailField	No	Unique	Unique login identifier
`shop_id`	ForeignKey	Yes	FK -> `tenants_shop`	Scoped tenant shop (null for platform admin)
`branch_id`	ForeignKey	Yes	FK -> `tenants_branch`	Scoped branch (null for all-branch managers)
`role`	CharField(20)	No	Enum (`OWNER` , `MANAGER` , `CASHIER` , `STAFF` , `RESELLER` , `PLATFORM_ADMIN`)	User role type
`phone`	CharField(20)	Yes		Contact mobile number
`is_staff`	BooleanField	No	Default `False`	Platform administration access
`last_seen`	DateTimeField	Yes		Timestamp of last user API activity

Table: `accounts_role` & `accounts_permission`

Granular role-based access control (RBAC).

TABLE	KEY COLUMNS	DESCRIPTION
`accounts_permission`	`code` (Unique), `name`, `category`, `description`	Global catalog of permissions (e.g. `create_sale`, `manage_expenses`)
`accounts_role`	`shop_id`, `name`, `role_type`, `is_system`	Tenant-defined customizable permission roles
`accounts_role_permissions`	`role_id` (FK), `permission_id` (FK)	Many-to-Many junction table for RBAC

3.3 App: `catalog`

Table: `catalog_product`

Master physical and service product inventory items.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Tenant owner
`name`	CharField(200)	No		Product title / model name
`sku`	CharField(100)	Yes	Index	Stock Keeping Unit
`barcode`	CharField(100)	Yes	Index	Primary retail barcode (EAN-13, Code128, etc.)
`category_id`	ForeignKey	Yes	FK -> `catalog_category`	Product category classification
`brand_id`	ForeignKey	Yes	FK -> `catalog_brand`	Brand / Manufacturer
`unit_id`	ForeignKey	Yes	FK -> `catalog_unit`	Unit of measure (Pcs, Box, Kg, Liter)
`supplier_id`	ForeignKey	Yes	FK -> `purchasing_supplier`	Default vendor
`cost_price`	DecimalField(12,2)	No	Default `0.00`	Latest unit purchase cost price
`selling_price`	DecimalField(12,2)	No	Default `0.00`	Standard retail selling price
`track_inventory`	BooleanField	No	Default `True`	If `False`, item is untracked service/fee
`current_stock`	DecimalField(12,2)	No	Default `0.00`	Cached on-hand inventory quantity
`reorder_level`	PositiveIntegerField	No	Default `5`	Safety stock threshold for alerts
`warranty_months`	PositiveSmallInt	No	Default `0`	Standard warranty duration in months

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`replacement_guarantee_days`	PositiveSmallInt	No	Default `0`	Replacement coverage in days

Table: `catalog_productunit`

HIGH *-precision serialized unit tracking (IMEI, unique serial numbers).*

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Tenant owner
`product_id`	ForeignKey	No	FK -> `catalog_product`	Parent catalog product
`barcode`	CharField(100)	No	Unique per product	Unique serial / IMEI barcode
`status`	CharField(20)	No	Enum (`IN_STOCK` , `SOLD` , `TESTING_PENDING` , `DEFECTIVE` , `RETURNED_SUPPLIER`)	Physical unit status
`cost_price`	DecimalField(12,2)	No		Snapshotted purchase cost of this unit
`selling_price`	DecimalField(12,2)	No		Snapshotted selling price
`sale_id`	ForeignKey	Yes	FK -> `sales_sale`	Linked sale invoice when sold
`sold_at`	DateTimeField	Yes		Timestamp when unit was sold

3.4 App: `inventory`

Table: `inventory_stockmovement`

Immutable, append-only warehouse stock ledger.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Tenant owner
`product_id`	ForeignKey	No	FK -> `catalog_product`	Modified product
`branch_id`	ForeignKey	Yes	FK -> `tenants_branch`	Branch location where stock changed
`movement_type`	CharField(30)	No	Enum (`PURCHASE_IN` , `SALE_OUT` , `SALE_RETURN_IN` , `ADJUSTMENT_IN` , `ADJUSTMENT_OUT` , `DAMAGE_OUT` , `TRANSFER_IN` , `TRANSFER_OUT` , `SERVICE_CONSUMED`)	Nature of stock change
`quantity`	DecimalField(12,2)	No		Positive or negative delta quantity
`unit_cost`	DecimalField(12,2)	No		Snapshotted unit cost at movement time
`reference_type`	CharField(50)	Yes		Source model name (e.g. `'Sale'` , `'PurchaseOrder'`)
`reference_id`	CharField(100)	Yes		Source transaction ID or invoice number
`created_by_id`	ForeignKey	Yes	FK -> `accounts_user`	Operator who executed the movement
`created_at`	DateTimeField	No	Auto Now Add	Movement execution timestamp

3.5 App: `sales` & `pos`

Table: `sales_sale`

Master sales invoice document.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Tenant owner
`invoice_no`	CharField(50)	No	Unique per shop	System generated memo number
`customer_id`	ForeignKey	Yes	FK -> `crm_customer`	Registered customer (or null for walk-in)
`customer_name`	CharField(150)	Yes		Walk-in buyer name
`customer_phone`	CharField(20)	Yes		Walk-in buyer contact number
`sale_date`	DateTimeField	No	Index	Timestamp of checkout
`subtotal`	DecimalField(12,2)	No	Default `0.00`	Gross sum of all items
`discount`	DecimalField(12,2)	No	Default `0.00`	Total discount deducted
`delivery_charge`	DecimalField(12,2)	No	Default `0.00`	Shipping / courier fee
`tax`	DecimalField(12,2)	No	Default `0.00`	Computed VAT amount
`total`	DecimalField(12,2)	No	Default `0.00`	Net payable: `subtotal - discount + delivery + tax`
`paid`	DecimalField(12,2)	No	Default `0.00`	Total cash/digital amount received
`status`	CharField(20)	No	Enum (`PAID` , `PARTIAL` , `DUE` , `CANCELLED` , `QUOTATION`)	Invoicing payment status
`idempotency_key`	CharField(100)	Yes	Unique	Prevents duplicate POS checkouts

Table: `sales_saleitem`

Individual product line items within a sales invoice.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`sale_id`	ForeignKey	No	FK -> `sales_sale`	Parent sales invoice
`product_id`	ForeignKey	No	FK -> `catalog_product`	Sold catalog product
`quantity`	DecimalField(12,2)	No		Quantity sold
`unit_price`	DecimalField(12,2)	No		Selling price charged per unit
`unit_cost`	DecimalField(12,2)	No		Snapshotted purchase cost (for COGS & profit)
`discount`	DecimalField(12,2)	No	Default `0.00`	Item level discount
`subtotal`	DecimalField(12,2)	No		Net line amount

Table: `sales_emischedule` & `sales_emiinstallment`

Financed installment sales plans.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`sale_id`	OneToOneField	No	FK -> `sales_sale`	Linked sale invoice
`customer_id`	ForeignKey	No	FK -> `crm_customer`	Financed customer
`total_emi_amount`	DecimalField(12,2)	No		Principal + total interest
`down_payment`	DecimalField(12,2)	No		Upfront initial cash paid
`interest_percent`	DecimalField(5,2)	No	Default `0.00`	Interest rate charged
`total_months`	PositiveSmallInt	No		Number of installment months
`monthly_installment`	DecimalField(12,2)	No		Monthly amount due
`status`	CharField(20)	No	Enum (`ACTIVE` , `COMPLETED` , `DEFAULTED`)	EMI schedule health

3.6 App: `accounting`

Table: `accounting_ledgerentry`

Append-only double-entry financial cash flow tracking table.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Tenant owner
`account`	CharField(20)	No	Enum (`CASH` , `BKASH` , `NAGAD` , `BANK` , `CARD` , `OTHER`)	Specific liquid account
`amount`	DecimalField(12,2)	No		Positive (Inflow) or Negative (Outflow)
`source_type`	CharField(50)	No		Event type (`SALE` , `EXPENSE` , `DUE_COLLECT` , `PURCHASE` , `TRANSFER` , `INVESTMENT` , `DRAWING`)
`source_id`	CharField(100)	Yes		Foreign source record ID
`description`	CharField(255)	Yes		Human-readable audit narrative
`created_at`	DateTimeField	No	Auto Now Add	Financial entry timestamp

Table: `accounting_dailysettlement`

Daily shift reconciliation and cash drawer closing audits.

COLUMN	TYPE	NULLABLE	CONSTRAINTS	DESCRIPTION
`id`	BigAutoField	No	PK	Primary Key
`shop_id`	ForeignKey	No	FK -> `tenants_shop`	Tenant owner
`opened_at`	DateTimeField	No		Shift opening timestamp
`closed_at`	DateTimeField	Yes		Shift closing timestamp
`opening_cash`	DecimalField(12,2)	No		Carryover cash from previous shift
`expected_cash`	DecimalField(12,2)	No		System calculated cash: `opening + sales - expenses`
`actual_cash`	DecimalField(12,2)	Yes		Manually counted cash in physical drawer
`discrepancy`	DecimalField(12,2)	Yes		Variance: `actual_cash - expected_cash`
`status`	CharField(20)	No	Enum (`OPEN` , `CLOSED`)	Shift register state

3.7 App: `service`

Table: `service_warranty` & `service_serviceticket`

Warranty lifecycle and repair center job card management.

TABLE	KEY COLUMNS	DESCRIPTION
`service_warranty`	`product_id`, `product_unit_id`, `customer_id`, `serial_no`, `period_months`, `start_date`, `expiry_date`, `status`	Warranty registration bound to individual serial units.
`service_serviceticket`	`ticket_no`, `customer_id`, `warranty_id`, `technician_id`, `device_type`, `issue_type`, `complaint`, `status`, `service_charge`, `discount`, `paid`	Service job card state machine (`RECEIVED` → `DIAGNOSING` → `AWAITING_PARTS` → `IN_REPAIR` → `READY` → `DELIVERED`).
`service_serviceticketpart`	`ticket_id`, `product_id`, `quantity`, `unit_cost`, `unit_price`, `from_stock`	Replacement spare parts used in repair with optional automated inventory deduction.

4. Key Database Constraints & Integrity Rules

```
-- 1. Strict Tenant Scoping Unique Constraints
ALTER TABLE catalog_product
ADD CONSTRAINT uniq_sku_per_shop_when_set
UNIQUE (shop_id, sku);

ALTER TABLE catalog_productunit
ADD CONSTRAINT uniq_unit_barcode_per_product
UNIQUE (shop_id, product_id, barcode);

ALTER TABLE sales_sale
ADD CONSTRAINT uniq_invoice_per_shop
UNIQUE (shop_id, invoice_no);

ALTER TABLE service_serviceticket
ADD CONSTRAINT uniq_ticket_no_per_shop
UNIQUE (shop_id, ticket_no);

-- 2. Non-Negative Financial Amounts (Check Constraints)
ALTER TABLE sales_payment
ADD CONSTRAINT payment_amount_positive
CHECK (amount > 0);

ALTER TABLE accounting_expense
ADD CONSTRAINT expense_amount_positive
CHECK (amount ⩾ 0);

ALTER TABLE accounting_investment
ADD CONSTRAINT investment_amount_positive
CHECK (amount ⩾ 0);
```

5. Summary Index Statistics

APP AREA	TOTAL MODELS	CORE DATABASE TABLES	KEY FOREIGN KEYS	UNIQUE CONSTRAINTS
Tenants & Core	4	4	6	3
Auth & Accounts	5	6	4	2
Catalog & Products	6	6	8	5
Inventory & Movements	1	1	3	1
CRM & Customers	2	2	2	1
Purchasing & Vendors	5	5	7	2
Sales, POS & EMI	7	7	10	4
Finance & Accounting	7	7	5	3
Service & Warranty	5	5	8	2
Platform & Billing	12	12	6	2
TOTALS	**54 Models**	**55 Tables**	**59 Relations**	**25 Constraints**